

Gate Futures WebSocket v4

Gate provides a simple and robust Websocket API to integrate Gate BTC/USDT futures contract trade status into your business or application.

We have language bindings in [Python](#) and [Golang](#), more in the future! You can view code examples in the dark area to the right, and you can switch the programming language of the examples with the tabs in the top right.

Server URL

We provide BTC/USDT futures contract trade server urls, you can choose one of them according to your condition.

BTC Contract

Base URLs:

- Real Trading: <wss://fx-ws.gateio.ws/v4/ws/btc>
- TestNet Trading: <wss://fx-ws-testnet.gateio.ws/v4/ws/btc>

USDT Contract

Base URLs:

- Real Trading: <wss://fx-ws.gateio.ws/v4/ws/usdt>
- TestNet: <wss://ws-testnet.gate.com/v4/ws/futures/usdt>

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

`-testnet.gateio.ws/v4/ws`), we will use BTC contract for you.

Changelog

2026-01-07

- Added field `market_order_slip_ratio` to `futures.orders` (preset slippage ratio for market orders)
- `futures.order_place` `futures.order_batch_place` added field `market_order_slip_ratio` (allows maximum slippage for custom market orders)

2025-12-09

- The number of contracts supports decimals, and all pushed contracts, trading volume, etc. are changed to strings (possibly decimals).
- How to connect to websocket decimal support:
- When requesting a link to a websocket again, add the header of `"X-Gate-Size-Decimal": "1"` , and the number of shares, trading volume, etc. will be a string (possibly a decimal).
 - If you do not add the header of `"X-Gate-Size-Decimal": "1"` when linking to a websocket, the push will keep the original field type (int).
 - Please switch to string push support as soon as possible, and the subsequent shaping push will be offline (the offline time will be notified separately).
 - If there is a decimal size, if the user is still using the int push, then the

Demo WebSocket application

```
py
# !/usr/bin/env python
# coding: utf-8

import hashlib
import hmac
import json
import logging
import time
import threading

from websocket import WebSocketApp

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

event = threading.Event()

class GateWebSocketApp(WebSocketApp):

    def __init__(self, url, api_key, api_secret):
        super(GateWebSocketApp, self).__init__(url)
        self.api_key = api_key
        self.api_secret = api_secret

    def _send_ping(self):
        while not event.wait(10):
            self.last_ping_tm = time.time()
            if self.sock:
                try:
                    self.sock.ping()
                except Exception as ex:
                    logger.warning("send_ping exception: %s", ex)
                    break
            try:
                self._request("futures.ping")
            except Exception as e:
                raise e
```

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

- The following push channels and corresponding fields will be affected.

channel	field
futures.trades	size
futures.tickers	total_size
futures.book_ticker	A B
futures.order_book_update	a.s、 b.s
futures.order_book	a.s、 b.s
futures.obu	size
futures.candlesticks	v
futures.liquidates	left、 size、 order_size
futures.public_liquidates	size
futures.contract_stats	long_liq_size、 short_liq_size、 open_interest
futures.orders	iceberg、 left、 size
futures.usertrades	size
futures.auto_deleverages	position_size、 trade_size
futures.positions	size
futures.autoorders	position_size、 trade_size

- The following API request channels will be affected, mainly involving fields

```

"time": current_time,
"channel": channel,
"event": event,
"payload": payload,
}
if auth_required:
    message = 'channel=%s&event=%s&payload=%s' % (channel, event, payload)
    data['auth'] = {
        "method": "api_key",
        "KEY": self._api_key,
        "SIGN": self.get_sign(message)
    }
    data = json.dumps(data)
    logger.info('request: %s', data)
    self.send(data)

def get_sign(self, message):
    h = hmac.new(self._api_secret.encode(), message.encode(), hashlib.sha256)
    return h.hexdigest()

def subscribe(self, channel, payload):
    self._request(channel, "subscribe", payload)

def unsubscribe(self, channel, payload):
    self._request(channel, "unsubscribe", payload)

def on_message(ws, message):
    # type: (GateWebSocketApp, str) -> None
    # handle message received
    logger.info("message received from %s", ws)

def on_open(ws):
    # type: (GateWebSocketApp) -> None
    # subscribe to channels interested
    logger.info('websocket connected')
    ws.subscribe("futures.tickers", [{"channel": "futures.tickers"}])

# custom header
custom_headers = {
    "X-Gate-Size-Decimal": "1"
}

if __name__ == "__main__":
    logging.basicConfig(format="%(asctime)s %(message)s", level=logging.INFO)
    app = GateWebSocketApp("wss://fx-gate.com/ws/v4/futures")
    app.set_api_key("YOUR_API_KEY")
    app.set_api_secret("YOUR_API_SECRET")
    app.on_open=on_open

```

Spot & Margin		Perpetual Futures	Delivery Futures	Options	Unifi
channel	field				
futures.order_place	size, left				
futures.order_batch_place	size				
futures.order_cancel	size, left				
futures.order_cancel_cp	size, left				
futures.order_amend	size, left				
futures.order_list	size, left				
futures.order_status	size, left				

2025-09-25

- Add `Positions ADL API` documentation

2025-05-22

- Added documentation for the Order Book V2 API
- Added a new field `full` to `futures.order_book_update`

2025-04-25

- Futures Account Trade has added a new `futures.order_cancel_ids` channel.
- `futures.order_book` and `futures.order_book_update` have added a new depth level field `1`.

2025-04-18

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

2025-03-24

- Fixed the incorrect descriptions in some documents of the order_book channel.
- Fixed the incorrect descriptions in some documents of the orders channel.

2025-03-21

- The documentation for channel `futures.orders` has been updated, with new explanations added for fields such as `update_id` , `update_time` , `biz_info` , `stop_profit_price` , `stop_loss_price` .

2025-03-12

- Add new field `contract` in channel `contract_stats`
- Updated the Futures Account Trade module and added the `x-gate-exptime` field
- Fixed some descriptive errors in the Futures Account Trade documentation

2025-02-19

- Added channel `futures.public_liquidates` to push liquidate orders snapshot

2025-02-10

- Updated the Futures Account Trade module and added `x_in_time` , `x_out_time` , `conn_trace_id` , `trace_id` fields
- `futures.order_place` , `futures.order_batch_place` , `futures.order_cancel` , `futures.order_cancel_cp` and `futures.order_amend` added `x_gate_ratelimit_requests_remain` , `x_gate_ratelimit_limit` and

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

2024-11-18

- remove `10` level and `1000ms` interval in channel `futures.order_book_update`

2023-09-21

- Add new field `is_internal` in channel `futures.trades`

2023-08-18

- Add WebSocket API
- WebSocket API allows placing, canceling, amending, querying orders through a WebSocket connection.

2023-07-07

- Add new interval `20ms` in channel `futures.order_book_update`, please note that the interval of `20ms` is only supported for `20` levels.

2023-06-20

- Add `update_id` of `futures.positions`

2022-12-22

- Add new field `auto_size` in `futures.autoorders` initial struct, field detail to http api

2022-11-22

- Add new field `time_ms` in common msg response for time of message created

2022-08-11

- Add new field `text` in user trades notification
- Add new fields `low_24h` and `high_24h` in tickers notification

2022-04-15

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

2021-03-31

- Add milliseconds field `t` in `futures.book_ticker` and `futures.order_book` response

2021-03-10

- Add new order book channel `futures.book_ticker` to push best ask/bid in realtime
- Add new order book channel `futures.order_book_update` to push order book change with user specified update frequency
- Add local order book maintenance documentation

2021-03-01

- Add new millisecond-precision timestamp ended with `_ms` in server notification.
- Add new field `id` in order book `all` notification.

2020-8-08

- Add a complete code demo(golang, python)

2020-8-07

- Add auto orders subscription

2020-7-07

- Add order book annotation

2020-4-30

- Add positions subscription

2019-11-06

- Add USDT futures contract
- Add `volume_24h_base` field, `volume_24h_settle` field, `volume_24h_quote` field for tickers
- Remove old server urls(`wss://fx-ws.gateio.ws/v4/ws` or `wss://fx-ws-`

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

If you use old server urls(`wss://fx-ws.gateio.ws/v4/ws` or `wss://fx-ws-testnet.gateio.ws/v4/ws`), we will use BTC contract for you.

2019-10-22

- Add application layer ping/pong message

2019-04-30

- Add index and mark candlestick subscription
- Add `funding_rate_indicative` field for tickers
- Add `is_reduce_only` and `status` field for orders

2019-02-13

- Change WebSocket base url
- Add `volume_24h_usd` field and `volume_24h_btc` field for tickers

2019-01-11

- Add `position_closes` and `balances` subscription
- Del `finish_time` field for `auto_deleverages` and `liquidates`
- Add `time` field for `auto_deleverages` and `liquidates`

API Overview

Method

Each general api (such as ticker, order book etc.) supports some different event messages, they are:

- `subscribe` (**RECOMMENDED TO USE**)

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

Server will not send new data notification if unsubscribed.

- **update**

If new subscribed data(incremental data) is available, server will send a notification to client.

- **all**

If new subscribed data(all data) is available, server will send a notification to client.

Request

Each request follows a common format, which contains **time** , **channel** , **event** and **payload** .

parameter	type	required	descripti
id	Integer	No	Optional back by t which rec
time	Integer	Yes	Request t
channel	String	Yes	Request s channel
auth	String	No	Request a section fo
event	String	Yes	Request e (subscrib
payload	Array	Yes	Request c

Response

field	type	required	descriptio
time	Integer	Yes	Response time
time_ms	Integer	Yes	Response time of millisecon
channel	String	Yes	Response channel
event	String	Yes	Response channel event (update/al
error	Object	Yes	Response error
result	Any	Yes	New data notificatio from the server, or response t client requests. Null if error i not null.

Note: type of `result` is channel specific if it's server-initiated data update notification, but response to client subscription request always set the `result` to `{"status": "success"}` . To verify if subscription request is successful or not, you only need to check if `error` field is null. Parsing `result` field is not necessary.

Channel specific description below will only give the server-initiated data update notification format for simplicity.

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

In case of error, you receive a message containing the proper error code and message within an error object.

Code	Message
1	invalid argument struct
2	invalid argument
3	service error
4	authentication fail

Authentication

Request body needs to carry authentication information if channels are private, e.g. `futures.usertrades`

WebSocket authentication uses the same signature calculation method with HTTP API, but has the following differences:

- Signature string concatenation method: `channel=<channel>&event=<event>&time=<time>`, where `<channel>`, `<event>`, `<time>` are corresponding request information
- Authentication information are sent in request body in field `auth`.

You can log into the console to retrieve futures API key and secret.

field	type	description
<code>method</code>	String	Allowed value: <code>api_key</code>
<code>KEY</code>	String	User key string

Code example

```

# example WebSocket signature calculation
import hmac, hashlib, json, time

def gen_sign(channel, event, timestamp):
    # GateAPIv4 key pair
    api_key = 'YOUR_API_KEY'
    api_secret = 'YOUR_API_SECRET'

    s = 'channel=%s&event=%s&time=%d' % (channel, event, timestamp)
    sign = hmac.new(api_secret.encode(), s.encode(), hashlib.sha256).hexdigest()
    return {'method': 'api_key', 'KEY': sign}

request = {
    'id': int(time.time() * 1e6),
    'time': int(time.time()),
    'channel': 'futures.orders',
    'event': 'subscribe',
    'payload': ["20011", "BTC_USDT"]
}
request['auth'] = gen_sign(request['channel'], request['event'], request['time'])
print(json.dumps(request))

```

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

SIGN

String

User sign string

System API

Provides system status check, such as ping-pong.

Ping and Pong

Check Server/Client connectivity.

Gate futures contract use the protocol layer ping/pong message. The server will initiate a ping message actively. If the client does not reply, the client will be disconnected.

[websocket rfc](#)

if you want to actively detect the connection status, you can send application layer ping message and receive pong message.

Request

- channel

futures.ping

Code example

```
from websocket import create_connection

ws = create_connection("wss://fx-ws-")
ws.send('{"time" : 123456, "channel" : "futures.ping"}')
print(ws.recv())
```

The above command returns JSON structured like this:

```
{
  "time": 1545404023,
  "time_ms": 1545404023123,
  "channel": "futures.pong",
  "event": "",
  "result": null
}
```

Tickers API

The ticker is a high level overview of the state of the contract. It shows you the highest, lowest, last trade price. It also includes information such as daily

		Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	result	Array	Array of objects	<pre>"volume_24h_btc": "117", "volume_24h_usd": "419950", "quanto_base_rate": "", "volume_24h_quote": "1665006", "volume_24h_settle": "178", "volume_24h_base": "5526", "low_24h": "99.2", "high_24h": "132.5" }] }</pre>		
	field		type	des		
	contract		String	Futu		
	last		String	Last		
	change_percentage		String	Cha		
	funding_rate		String	Fun		
	funding_rate_indicative		String	Indi rate (dep func		
	mark_price		String	Rec		
	index_price		String	Inde		
	total_size		String	Tota		
	volume_24h		String	Volu		
	quanto_base_rate		String	Excl curr sett Qua Doe con type		
	volume_24h_btc		String	Trac rece BTC use vo: vo: inst		



	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	volume_24h_usd	String	Trac rece USE use vo: vo: inst		
	volume_24h_quote	String	Trac rece curr		
	volume_24h_settle	String	Trac rece curr		
	volume_24h_base	String	Trac rece curr		
	low_24h	String	Low in re		
	high_24h	String	Hig in re		

Cancel subscription

Unsubscribe contract ticker.

Request

- channel
 - futures.tickers
- event
 - unsubscribe

Code example

```
import json
from websocket import create_connection

ws = create_connection("wss://fx-ws-...")
req = {
    "time": 123456,
    "channel": "futures.tickers",
    "event": "unsubscribe",
    "payload": ["BTC_USDT"]
}
```

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

The above command returns JSON structured like this:

```
json
{
  "time": 1545404900,
  "time_ms": 1545404900123,
  "channel": "futures.tickers",
  "event": "unsubscribe",
  "result": {
    "status": "success"
  }
}
```

Trades API

This channel sends a trade message whenever a trade occurs at Gate. It includes details of the trade, such as price, amount, time and type.

Trades subscription

Subscribe trades update notification.

Request

- channel

```
futures.trades
```

- event

```
subscribe
```

- params

Code example

```
py
from websocket import create_connection

ws = create_connection("wss://fx-ws-1.gate.io")
ws.send('{"time" : 123456, "channel": "futures.trades", "event": "subscribe"}')
print(ws.recv())
```

The above command returns JSON structured like this:

```
json
{
  "time": 1545405058,
  "time_ms": 1545405058123,
  "channel": "futures.trades",
  "event": "subscribe",
  "result": {
    "status": "success"
  }
}
```

		Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	payload	Array	Yes	Contract list	<pre>"status": "success" }</pre>	

Trades notification

Notify latest trades update.

Positive size means taker is buyer, negative seller

Notify

- channel
 - futures.trades
- event
 - update
- params

field	type	description
result	Array	Array of objects

field	type	description
contract	string	Futures contract name
size	string/int	Trades size
id	int	Trades id
create_time	int	Trades ms create time
create_time_ms	int	Trades ms create time in millisecond

```
json
{
  "channel": "futures.trades",
  "event": "update",
  "time": 1541503698,
  "time_ms": 1541503698123,
  "result": [
    {
      "size": "-108",
      "id": 27753479,
      "create_time": 1545136464,
      "create_time_ms": 1545136464123,
      "price": "96.4",
      "contract": "BTC_USDT",
      "is_internal": true
    }
  ]
}
```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	price	string	Trades price		
is_internal	bool	Whether internal trade. Internal trade refers to the takeover and liquidation orders by the insurance fund and ADL users. Since it is not a normal matching on the market depth, the transaction price may deviate, and it will not be recorded in the K-line. If it is not an internal trade, this field will not be returned.			

Cancel subscription

Unsubscribe trades update notification.

Request

- channel

Code example

```
from websocket import create_connection

ws = create_connection("wss://fx-ws-...")
ws.send("...")
```

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

unsubscribe

The above command returns JSON structured like this:

```

{
  "time": 1545404900,
  "time_ms": 1545404900123,
  "channel": "futures.trades",
  "event": "unsubscribe",
  "result": {
    "status": "success"
  }
}

```

Order Book API

The `order_book` channel allow you to keep track of the state of the Gate order book depth. It is provided on a price aggregated basis, with customizable precision.

There are three different order book channels for subscription:

- `futures.order_book`

Legacy channel, which uses `all` to push full limited-level order book, and `update` to send every order book change event.

- `futures.book_ticker`

Push best bid and ask in real-time.

- `futures.order_book_update`

Push order book change with user-specified update frequency.

Receiving order book update through `futures.order_book` is not

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

How to maintain local order book:

- Subscribe `futures.order_book_update` with specified level and update frequency, e.g. `["BTC_USDT", "100ms", "100"]` pushes the first 100 levels' update in BTC_USDT order book every 100ms
- Cache WebSocket notifications. Every notification use `U` and `u` to tell the first and last update ID since last notification.
- Retrieve base order book using REST API, and make sure the order book ID is recorded(referred as `baseID` below) e.g.
`https://api.gateio.ws/api/v4/futures/usdt/ocontract=BTC_USDT&limit=10&with_id=true`
retrieves the 10-level base order book of BTC_USDT
- Iterate the cached WebSocket notifications, and find the first one which contains the baseID, i.e. `U <= baseID+1` and `u >= baseID+1`, then start consuming from it. Note that sizes in notifications are all absolute values. Use them to replace original sizes in corresponding price. If size equals to 0, delete the price from the order book.
- Dump all notifications which satisfy `u < baseID+1`. If `baseID+1 < first notification U`, it means current base order book falls behind notifications. Start from step 3 to retrieve newer base order book.
- If any subsequent notification which satisfy `U > baseID+1` is found, it means some updates are lost. Reconstruct local order book from step 3.

Note:

local order book's `id` and `timestam`
`p` to ensure consistency with the
server's state.

- The subscribed `level` (depth) in
WebSocket should match the `limit`
parameter in the REST snapshot. A
mismatch may result in incremental
updates failing to fully cover the
snapshot levels, leading to misaligned
or incomplete local order books.

Legacy order book subscription

Subscribe order_book.

Request

- channel
 - `futures.order_book`
- event
 - `subscribe`
- params

parameter	type	required	descri
<code>contract</code>	String	Yes	Contra name
<code>limit</code>	String	Yes	Depth 100, 50, 10, 5, 1

Code example

```
from websocket import create_connection

ws = create_connection("wss://fx-ws-1")
ws.send('{"time" : 123456, "channel": "futures.order_book", "event": "subscribe"}')
print(ws.recv())
```

The above command returns JSON
structured like this:

```
{
  "time": 1545405058,
  "time_ms": 1545405058123,
  "channel": "futures.order_book",
  "event": "subscribe",
  "result": {
    "status": "success"
  }
}
```

		Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	interval	String	Yes	Legal interval "0"		

Legacy order book notification

Notify contract order book update information

Notify

- channel
 - futures.order_book
- event
 - all
- params

field	type	description
result	object	Order book info
» t	Integer	Order book generation timestamp in milliseconds
» contract	String	Contract name
» id	Integer	OrderBook id
» asks	Array	OrderBook ask list

json

```
{
  "channel": "futures.order_book",
  "event": "all",
  "time": 1541500161,
  "time_ms": 1541500161123,
  "result": {
    "t": 1541500161123,
    "contract": "BTC_USD",
    "id": 93973511,
    "asks": [
      {
        "p": "97.1",
        "s": "2245"
      },
      {
        "p": "97.1",
        "s": "2245"
      }
    ],
    "bids": [
      {
        "p": "97.1",
        "s": "2245"
      },
      {
        "p": "97.1",
        "s": "2245"
      }
    ],
    "l": "20"
  }
}
```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	»» p	String	Ask price		
	»» s	String	Ask size		
	» bids	Array	OrderBook bid List		
	»» p	String	Bid price		
	»» s	String	Bid size		
	» l	String	Depth level (For example, 100 represents a depth update of 100 levels)		

Legacy order book unsubscription

Unsubscribe specified contract order book.

Request

- channel
 - futures.order_book
- event
 - unsubscribe

Code example

```
from websocket import create_connection

ws = create_connection("wss://fx-ws-...")
ws.send('{"time" : 123456, "channel": "futures.order_book", "event": "unsubscribe"}')
print(ws.recv())
```

The above command returns JSON structured like this:

```
{
  "time": 1545445847,
  "time_ms": 1545445847123,
  "channel": "futures.order_book",
  "event": "unsubscribe",
  "result": {
```

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

Best ask/bid subscription

Subscribe `book_ticker`.

Request

- channel

`futures.book_ticker`

- event

`subscribe`

- params

`payload` is an array contains contracts interested.

Code example

```
from websocket import create_connection

ws = create_connection("wss://fx-ws.gate.io")
ws.send('{"time" : 123456, "channel": "futures.book_ticker", "event": "subscribe"}')
print(ws.recv())
```

The above command returns JSON structured like this:

```
{
  "time": 1545405058,
  "time_ms": 1545405058123,
  "channel": "futures.book_ticker",
  "event": "subscribe",
  "result": {
    "status": "success"
  }
}
```

Best ask/bid notification

If `a` is empty string, it means empty asks; if `b` is empty string, it means empty bids.

Notify contract order book best bid and ask

Notify

- channel

```
{
  "time": 1615366379,
  "time_ms": 1615366379123,
  "channel": "futures.book_ticker",
  "event": "update",
  "result": {
    "t": 1615366379123,
    "u": 2517661076,
    "s": "BTC_USD",
  }
}
```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
update	"A": "47061" } }				
◦ params					
	field	type	description		
	result	object	Best bid and ask		
	» t	Integer	Book ticker generated timestamp in milliseconds		
	» u	String	Order book update ID		
	» s	String	Contract name		
	» b	String	Best bid price. If no bids, it's empty string		
	» B	String/Integer	Best bid size. If no bids, it will be 0		
	» a	String	Best ask price. If no asks, it's empty string		
	» A	String/Integer	Best ask size. If no asks, it will be 0		

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

Best ask/bid unsubscription

Unsubscribe specified contract order book.

Request

- channel

futures.book_ticker

- event

unsubscribe

Code example

```
from websocket import create_connection  
  
ws = create_connection("wss://fx-ws.gate.io")  
ws.send('{"time" : 123456, "channel": "futures.book_ticker"}')  
print(ws.recv())
```

The above command returns JSON structured like this:

```
{  
  "time": 1545445847,  
  "time_ms": 1545445847123,  
  "channel": "futures.book_ticker",  
  "event": "unsubscribe",  
  "result": {  
    "status": "success"  
  }  
}
```

Order book update subscription

Subscribe order_book_update.

Request

- channel

futures.order_book_update

- event

subscribe

- params

Code example

```
from websocket import create_connection  
  
ws = create_connection("wss://fx-ws.gate.io")  
ws.send('{"time" : 123456, "channel": "futures.order_book_update"}')  
print(ws.recv())
```

The above command returns JSON structured like this:

		Spot & Margin		Perpetual Futures	Delivery Futures	Options	Unifi
	contract	String	Yes	Contr name	<pre>"time_ms": 1545405058123, "channel": "futures.order_book_upd "event": "subscribe", "result": { "status": "success" } }</pre>		
	frequency	String	Yes	Upda frequ 20ms 100m			
	level	String	No	Optic depth intere Only updat withir notifi Allow value "100" "20"; 20ms Allow "20"			

Order book update notification

Notify contract order book update

Notify

- channel
 - futures.order_book_update
- event
 - update
- params

```
json  
  
{  
  "time": 1615366381,  
  "time_ms": 1615366381123,  
  "channel": "futures.order_book_upd  
  "event": "update",  
  "result": {  
    "t": 1615366381417,  
    "s": "BTC_USD",  
    "U": 2517661101,  
    "u": 2517661113,  
    "b": [  
      {  
        "p": "54672.1",  
        "s": "0"  
      },  
      ]  
    }  
  }  
}
```

		Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	result	Object	Changed asks and bids since last update	<pre>}], "a": [{ "p": "54743.6", "s": "0" }, { "p": "54742", "s": "95" }], "l": "100" }</pre>		
	» t	Integer	Order book generation timestamp in milliseconds			
	» full	Boolean	true indicates a full depth snapshot (e.g., if the subscribed level is 100, the pushed data will contain 100 levels of depth). Upon receiving this, the user should replace the local order book with the provided data. false indicates incremental depth updates. When the value is false , this field is omitted from the message			

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

» s	String	Contract name
» U	Integer	First order book update ID since last update
» u	Integer	Last order book update ID since last update
» b	Array	Changed bids
»» p	String	Changed price
»» s	String/Integer	Absolute size value after change. If 0, remove this price from order book
» a	Array	Changed asks
»» p	String	Changed price
»» s	String/Integer	Absolute size value after change. If 0, remove this price from order book
» l	String	Depth level (For

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi

Order book update unsubscribe

Unsubscribe specified contract order book.

Request

- channel
 - futures.order_book_update
- event
 - unsubscribe

Code example

```
from websocket import create_connection

ws = create_connection("wss://fx-ws.gate.io")
ws.send('{"time": 123456, "channel": "futures.order_book_update", "event": "unsubscribe"}')
print(ws.recv())
```

The above command returns JSON structured like this:

```
{
  "time": 1545445847,
  "time_ms": 1545445847123,
  "channel": "futures.order_book_update",
  "event": "unsubscribe",
  "result": {
    "status": "success"
  }
}
```

Order Book V2 API

Provide a faster method for retrieving

Maintain local depth

Note:

- Full Depth Snapshot Push
(`full=true`):When the channel pushes full depth data, the local depth should be completely replaced with the received data. Additionally, the `depth ID` should be updated to the value of the `u` field in the message. Note that the server may send full depth snapshots multiple times.
 - The first message pushed after subscribing to this channel is a full depth snapshot.
- Incremental Depth Push
(`full=false`):Incremental messages do not include the full field. Instead, each message contains the fields `u` (starting depth ID) and `u` (ending depth ID).
 - If `u` equals the local `depth ID` + `1` , it indicates a continuous depth update:
 - Replace the local depth ID with the value of `u` from the message.
 - If `a` (asks) and `b` (bids) in the update are not empty, update the corresponding bid and ask depth levels based on price.(Each `level[0]` represents the price, and `level[1]` represents the quantity.)If `level[1]` is `"0"` , the corresponding price level should be removed from the order book.
 - If `u` $\neq$ local `depth ID` + `1` , the depth data is not continuous. In this case, you must unsubscribe from the market and resubscribe to retrieve the initial depth snapshot.

```

{
  "time": 1747391482,
  "time_ms": 1747391482960,
  "id": 1,
  "conn_id": "d9db9373dc5e081e",
  "trace_id": "ee001938590e183db957b0",
  "channel": "futures.obu",
  "event": "subscribe",
  "payload": [
    "ob.BTC_USDT.400"
  ],
  "error": {
    "code": 2,
    "message": "Alert sub ob.BTC_USDT"
  },
  "result": {
    "status": "fail"
  }
}

```

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

only once. Duplicate subscription attempts will result in an error.
Example of a failed attempt:

Order book V2 subscription

Request

- channel

`futures.obu`

- event

`subscribe`

- params

`payload` It is a list containing stream names. The format is: `ob.{symbol}. {level}`; for example, `ob.BTC_USDT.400`, `ob.BTC_USDT.50`

The `level` enum values are: 400, 50; Update frequency: 400-level updates every 100 ms; 50-level updates every 20 ms.

Code example

```
from websocket import create_connection

ws = create_connection("wss://fx-ws-...")
ws.send('{"time" : 123456, "channel": "futures.obu", "event": "subscribe", "payload": ["ob.BTC_USDT.400"]}')
print(ws.recv())
```

The above command returns JSON structured like this:

```
{
  "time": 1747391482,
  "time_ms": 1747391482384,
  "id": 1,
  "conn_id": "d9db9373dc5e081e",
  "trace_id": "ee001938590e183db957b...",
  "channel": "futures.obu",
  "event": "subscribe",
  "payload": [
    "ob.BTC_USDT.400"
  ],
  "result": {
    "status": "success"
  }
}
```

Order book V2 update notification

Notify

- channel
 - futures.obu
- event
 - update
- params

field	type
result	Object
» t	Integer
» full	Boolean
» s	String
» u	Integer

```
{
  "channel": "futures.obu",
  "result": {
    "t": 1743673026995,
    "full": true,
    "s": "ob.BTC_USDT.400",
    "u": 79072179673,
    "b": [
      ["83705.9", "30166"]
    ],
    "a": [
      ["83706", "4208"]
    ]
  },
  "time_ms": 1743673026999
}
```

Example of incremental push:

json

```
{
  "channel": "futures.obu",
  "result": {
    "t": 1743673027017,
    "s": "ob.BTC_USDT.400",
    "U": 79072179674,
    "u": 79072179694,
    "b": [
      ["83702.2", "62"],
      ["83702.1", "0"],
      ["83702", "0"],
      ["83685.6", "120"],
      ["83685", "239"]
    ]
  },
  "time_ms": 1743673027020
}
```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
» u	Integer				
» b	Array[OrderBookArray]				
»» OrderBookArray	Array[String]				
» a	Array[OrderBookArray]				
»» OrderBookArray	Array[String]				

Order book V2 update
unsubscription

Unsubscribe specified contract order
book v2

Code example

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	- channel - futures.obu - event - unsubscribe		<pre>ws = create_connection("wss://fx-ws-1") ws.send('{"time" : 123456, "channel" : "futures.obu"}') print(ws.recv())</pre>		

The above command returns JSON structured like this:

```
json
{
  "time": 1743673617,
  "time_ms": 1743673617242,
  "id": 1,
  "conn_id": "7b06ff199a98ab0e",
  "trace_id": "8f86e4021a84440e502f73",
  "channel": "futures.obu",
  "event": "unsubscribe",
  "payload": ["ob.BTC_USDT.400"],
  "result": {
    "status": "success"
  }
}
```

Candlesticks API

Provides a way to access charting candlestick info.

Candlesticks subscription

If prefix contract with mark_ , the contract's mark price candlesticks will be subscribed; if prefix with index_ , index price candlesticks will be subscribed.

Request

- channel

Code example

```
python
from websocket import create_connection

ws = create_connection("wss://fx-ws-1")
ws.send('{"time" : 123456, "channel" : "contract.mark.BTC_USDT.400"}')
print(ws.recv())
```

subscribe

- params

field	type	description
interval	String	Interval : "10s", "1m", "5m", "15m", "30m", "1h", "4h", "8h", "1d", "7d"
contract	String	Futures contract name

json

```
{
  "time": 1545445847,
  "time_ms": 1545445847123,
  "channel": "futures.candlesticks",
  "event": "subscribe",
  "result": {
    "status": "success"
  }
}
```

Candlesticks notification

Notify kline information of subscribed contract.

Notify

- channel

futures.candlesticks

- event

update

- params

field	type	description
result	Array	Array of objects

json

```
{
  "time": 1542162490,
  "time_ms": 1542162490123,
  "channel": "futures.candlesticks",
  "event": "update",
  "result": [
    {
      "t": 1545129300,
      "v": "27525555",
      "c": "95.4",
      "h": "96.9",
      "l": "89.5",
      "o": "94.3",
      "n": "1m_BTC_USD",
      "a": "314732.87412",
      "w": false
    },
    {
      "t": 1545129300,
      "v": "27525555",
      "c": "95.4",
      "h": "96.9",

```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
t	Integer	Time	<pre>"a": "314732.87412", "w": true }] }</pre>		
o	String	Open			
c	String	Close			
h	String	Highest			
l	String	Lowest			
v	String/Integer	Volume			
n	String	Futures contract name			
a	String	Amount			
w	Boolean	true means window close. true may be missing, but does not affect data usage			

Cancel subscription

Unsubscribe specified contract kline information.

Request

- channel
 - futures.candlesticks
- event
 - unsubscribe

Code example

```
from websocket import create_connection

ws = create_connection("wss://fx-ws-...")
ws.send('{"time" : 123456, "channel" : "futures.candlesticks"}')
print(ws.recv())
```

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

```

{
  "time": 1545445847,
  "time_ms": 1545445847123,
  "channel": "futures.candlesticks",
  "event": "unsubscribe",
  "result": {
    "status": "success"
  }
}

```

public liquidates order API

Provide a way to receive Gate liquidate order information, and each contract can push up to one liquidate order data every 1 second

public liquidate order subscription

If you want to subscribe to liquidates order push in all contracts, use `!all` in the subscription request list

public liquidates order notify

request parameters

- channel `futures.public_liquidates`
- event `subscribe`
- params

Code example

```

import json
from websocket import create_connection

ws = create_connection("wss://fx-ws-1.gate.io")
req = {
  "time": 123456,
  "channel": "futures.public_liquidates",
  "event": "subscribe",
  "payload": ["BTC_USD", "ETH_USD"]
}
ws.send(json.dumps(req))
print(ws.recv())

```

		Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	contract	String	yes	contra list	json <pre>{ "time": 1545459681, "time_ms": 1545459681123, "channel": "futures.public_liquidate", "event": "subscribe", "result": { "status": "success" } }</pre>	

public liquidate order notification

Push public liquidate order updates

Notify params

- channel
 - futures.public_liquidates
- event
 - update
- params

name	type	describe
result	Array	Array of objects

parameter	type	description
price	Float	order price
size	string/Integer	liquidate order quantity
time_ms	Integer	time_in_mill

json

```
{
  "channel": "futures.public_liquidate",
  "event": "update",
  "time": 1541505434,
  "time_ms": 1541505434123,
  "result": [
    {
      "price": 215.1,
      "size": "-124",
      "time_ms": 1541486601123,
      "contract": "BTC_USD",
    }
  ]
}
```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	contract	string	contract		

Cancel subscription

public liquidate order Unsubscribe

params

- channel
 - futures.public_liquidates
- event
 - unsubscribe

Code example

```
import json
from websocket import create_connection

ws = create_connection("wss://fx-ws-...")
req = {
    "time": 123456,
    "channel": "futures.public_liquidate",
    "event": "unsubscribe",
    "payload": ["BTC_USD"],
}
ws.send(json.dumps(req))
print(ws.recv())
```

The above command returns the JSON structure as follows:

```
{
  "time": 1545459681,
  "time_ms": 1545459681123,
  "channel": "futures.public_liquidate",
  "event": "unsubscribe",
  "result": {
    "status": "success"
  }
}
```

Contract Stats API

The contract_stats channel allows you

Contract Stats subscription

Subscribe futures contract stats.

Request

- channel
 - futures.contract_stats
- event
 - subscribe
- params

parameter	type	required	descri
contract	String	Yes	Future contra name
interval	String	Yes	Interva "1m", "15m", "30m", "1h", "8h", "3d", "

Code example

```
from websocket import create_connection

ws = create_connection("wss://fx-ws-...")
ws.send('{"time" : 123456, "channel": "futures.contract_stats", "event": "subscribe"}')
print(ws.recv())
```

The above command returns JSON structured like this:

```
{
  "time": 1545404023,
  "time_ms": 1545404023123,
  "channel": "futures.contract_stats",
  "event": "subscribe",
  "result": {
    "status": "success"
  }
}
```

Contract Stats notification

Notify subscribed contract stats.

Notify

- channel
 - futures.contract_stats
- event
 - update

```
{
  "time": 1541659086,
  "time_ms": 1541659086123,
  "channel": "futures.contract_stats",
  "event": "update",
  "result": [
    {
      "time": 1603865400,
```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi																														
	<table><tr><th>field</th><th>type</th><th>description</th></tr><tr><td>result</td><td>Array</td><td>Array of objects</td></tr></table>	field	type	description	result	Array	Array of objects																												
field	type	description																																	
result	Array	Array of objects																																	
	<table><tr><th>field</th><th>type</th><th>description</th></tr><tr><td>time</td><td>Integer</td><td>Start time of the liquidation</td></tr><tr><td>contract</td><td>String</td><td>Contract name</td></tr><tr><td>mark_price</td><td>Float</td><td>Current mark price</td></tr><tr><td>lsr_taker</td><td>Float</td><td>Long-short ratio of the taker</td></tr><tr><td>lsr_account</td><td>Float</td><td>Long-short ratio of the account</td></tr><tr><td>long_liq_size</td><td>String/Integer</td><td>Long liquidation size</td></tr><tr><td>long_liq_amount</td><td>Float</td><td>Long liquidation amount</td></tr><tr><td>long_liq_usd</td><td>Float</td><td>Long liquidation value</td></tr><tr><td>short_liq_size</td><td>String/Integer</td><td>Short liquidation size</td></tr></table>	field	type	description	time	Integer	Start time of the liquidation	contract	String	Contract name	mark_price	Float	Current mark price	lsr_taker	Float	Long-short ratio of the taker	lsr_account	Float	Long-short ratio of the account	long_liq_size	String/Integer	Long liquidation size	long_liq_amount	Float	Long liquidation amount	long_liq_usd	Float	Long liquidation value	short_liq_size	String/Integer	Short liquidation size		<pre>"long_liq_size": "0", "short_liq_size": "0", "open_interest": "124724", "short_liq_usd": 0, "mark_price": "8865", "top_lsr_size": 1.02, "short_liq_amount": 0, "long_liq_amount": 0, "open_interest_usd": 1511, "top_lsr_account": 1.5, "long_liq_usd": 0 }]</pre>		
field	type	description																																	
time	Integer	Start time of the liquidation																																	
contract	String	Contract name																																	
mark_price	Float	Current mark price																																	
lsr_taker	Float	Long-short ratio of the taker																																	
lsr_account	Float	Long-short ratio of the account																																	
long_liq_size	String/Integer	Long liquidation size																																	
long_liq_amount	Float	Long liquidation amount																																	
long_liq_usd	Float	Long liquidation value																																	
short_liq_size	String/Integer	Short liquidation size																																	

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	<code>short_liq_amount</code>	Float	Str lic ar cu		
	<code>short_liq_usd</code>	Float	Str lic vc cu		
	<code>open_interest</code>	String/Integer	O siz		
	<code>open_interest_usd</code>	Float	O vc cu		
	<code>top_lsr_account</code>	Float	To lo ac		
	<code>top_lsr_size</code>	Float	To lo pc		

Cancel subscription

Unsubscribe contract stats.

Request

- channel
 - `futures.contract_stats`
- event
 - `unsubscribe`
- params

Code example

```
import json
from websocket import create_connection

ws = create_connection("wss://fx-ws-1.gate.io")

req = {
    "time": 123456,
    "channel": "futures.contract_stats",
    "event": "unsubscribe",
    "payload": ["BTC_USD", "1m"]
}
```

		Spot & Margin		Perpetual Futures	Delivery Futures	Options	Unifi
	contract	String	Yes	Future contract name	The above command returns JSON structured like this:		
	interval	String	Yes	Interval: "1m", "15m", "30m", "1h", "8h", "7d"	<pre>json { "time": 1545404900, "time_ms": 1545404900123, "channel": "futures.contract_stats", "event": "unsubscribe", "result": { "status": "success" } }</pre>		

Note: contract is unsub_all , which means cancel all

Orders API

Provides a way to receive user closed orders.

Authentication required.

Orders subscription

If you want to subscribe to order updates in all contracts, use !all in contract list.

Subscribe user orders update

Request

- channel
 - futures.orders
- event
 - subscribe
- params

Code example

```
py
import json, time
from websocket import create_connection

ws = create_connection("wss://fx-ws-...")
req = {
  "time": int(time.time()),
  "channel": "futures.orders",
  "event": "subscribe",
  "payload": ["20011", "BTC_USD"],
  "auth": {
    "method": "api_key",
    "KEY": "xxxx",
    "SIGN": "xxxx"
```

		Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	user id	String	yes	User ic	print(ws.recv())	
	contract	String	yes	Future contra name	The above command returns JSON structured like this:	

json

```
{
  "time": 1545459681,
  "time_ms": 1545459681123,
  "channel": "futures.orders",
  "event": "subscribe",
  "result": {
    "status": "success"
  }
}
```

Orders notification

Notify user orders information when an order is put, updated or finished.

Notify

- channel
 - futures.orders
- event
 - update

- params
 - For the meaning of parameters, please refer to http interface.

field	type	description
result	Array	Array of objects

json

```
{
  "channel": "futures.orders",
  "event": "update",
  "time": 1541505434,
  "time_ms": 1541505434123,
  "result": [
    {
      "contract": "BTC_USD",
      "create_time": 1628736847,
      "create_time_ms": 1628736847329,
      "fill_price": 40000.4,
      "finish_as": "filled",
      "finish_time": 1628736848,
      "finish_time_ms": 1628736848320,
      "iceberg": "0",
      "id": 4872460,
      "is_close": false,
      "is_liq": false,
      "is_reduce_only": false,
      "left": "0",
      "mkfr": -0.00025,
      "price": 40000.4,
      "refr": 0,
      "refu": 0,
      "size": "1",
    }
  ]
}
```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	create_time	Integer	<pre>"tkfr": 0.0005, "user": "110xxxxx", "update_id": 1, "update_time": 1541505434123, "stop_loss_price": "", "stop_profit_price": "", "market_order_slip_ratio": "0.0" }] }</pre>		
	create_time_ms	Integer			
	fill_price	Float			
	finish_as	String			
	iceberg	String/Integer			

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
id		Integer			
is_close		Bool			
is_liq		Bool			
left		String/Integer			
mkfr		Float			
is_reduce_only		Bool			
status		String			
tkfr		Float			
price		Float			
refu		Integer			
refr		Float			
size		String/Integer			
text		String			

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	tif	String			
	finish_time	Integer			
	finish_time_ms	Integer			
	user	String			
	contract	String			
	stp_id	String			

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	update_time	Integer			
	biz_info	String			
	stop_profit_price	String			
	stop_loss_price	String			
	market_order_slip_ratio	String			

Cancel subscription

Unsubscribe user orders update notification, for all contract.

Request

- channel
 - futures.orders
- event
 - unsubscribe

Code example

```
py
import json, time
from websocket import create_connection

ws = create_connection("wss://fx-ws-...")
req = {
    "time": int(time.time()),
    "channel": "futures.orders",
    "event": "unsubscribe",
    "payload": ["20011", "BTC_USD"],
    "auth": {
        "method": "api_key",
        "KEY": "xxxx",
        "SIGN": "xxxx"
    }
}
```

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

The above command returns JSON structured like this:

```
json
{
  "time": 1545459681,
  "time_ms": 1545459681123,
  "channel": "futures.orders",
  "event": "unsubscribe",
  "result": {
    "status": "success"
  }
}
```

User trades API

Provides a way to receive user trades.

Authentication required.

User trades subscription

If you want to subscribe to user trade updates in all contracts, use `!all` in contract list.

Subscribe for user trades update.

Request

- channel
 - `futures.usertrades`
- event
 - `subscribe`
- params

Code example

```
py
import json, time
from websocket import create_connection

ws = create_connection("wss://fx-ws-1.gate.io")

req = {
  "time": int(time.time()),
  "channel": "futures.usertrades",
  "event": "subscribe",
  "payload": ["20011", "BTC_USD"],
  "auth": {
    "method": "api_key",
    "KEY": "xxxx",
    "SIGN": "xxxx"
  }
}
```

		Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	user id	String	yes	User ic		
	contract	String	yes	Future contra name		

The above command returns JSON structured like this:

json

```
{
  "time": 1545459681,
  "time_ms": 1545459681123,
  "channel": "futures.usertrades",
  "event": "subscribe",
  "result": {
    "status": "success"
  }
}
```

User trades notification

Notify user trades update.

Notify

- channel
 - futures.usertrades
- event
 - update
- params

field	type	description
result	Array	Array of objects

field	type	descri
contract	String	Futur
create_time	Integer	Creat

json

```
{
  "time": 1543205083,
  "time_ms": 1543205083123,
  "channel": "futures.usertrades",
  "event": "update",
  "result": [
    {
      "id": "3335259",
      "create_time": 1628736848,
      "create_time_ms": 1628736848320,
      "contract": "BTC_USD",
      "order_id": "4872460",
      "size": "1",
      "price": "40000.4",
      "role": "maker",
      "text": "api",
      "fee": 0.0009290592,
      "point_fee": 0
    }
  ]
}
```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
create_time_ms	Integer	Creat millis			
id	String	Trade			
order_id	String	Orde			
price	String	Price			
size	String/Integer	Trade			
role	String	User			
text	String	Orde Infor use tl custo defin meet cond 1. Mu 2. Exc lengt 28 by 3. Inp only letter hyph Besid infor follow reser indica sourc - web - api: - app - aut Auto - liqu classi force			



<https://www.gate.com/docs/developers/futures/ws/en/>

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

Request

- channel
`futures.usertrades`
- event
`unsubscribe`

```
import json, time
from websocket import create_connection

ws = create_connection("wss://fx-ws-1.gate.io")
req = {
    "time": int(time.time()),
    "channel": "futures.usertrades",
    "event": "unsubscribe",
    "payload": ["20011", "BTC_USD"],
    "auth": {
        "method": "api_key",
        "KEY": "xxxx",
        "SIGN": "xxxx"
    }
}
ws.send(json.dumps(req))
print(ws.recv())
```

The above command returns JSON structured like this:

```
json
{
  "time": 1545459681,
  "time_ms": 1545459681123,
  "channel": "futures.usertrades",
  "event": "unsubscribe",
  "result": {
    "status": "success"
  }
}
```

Liquidates API

Provides a way to receive user liquidates info.

Authentication required.

Liquidates subscription

If you want to subscribe to liquidate updates in all contracts, use `!all` in contract list.

Subscribe for user liquidates update.

Request

- channel
 - `futures.liquidates`
- event
 - `subscribe`
- params

parameter	type	required	descri
<code>user id</code>	String	yes	User ic
<code>contract</code>	String	yes	Future contra name

Code example

```
import json, time
from websocket import create_connection

ws = create_connection("wss://fx-ws-...")

req = {
    "time": int(time.time()),
    "channel": "futures.liquidates",
    "event": "subscribe",
    "payload": ["20011", "BTC_USD"],
    "auth": {
        "method": "api_key",
        "KEY": "xxxx",
        "SIGN": "xxxx"
    }
}

ws.send(json.dumps(req))
print(ws.recv())
```

The above command returns JSON structured like this:

```
{
  "time": 1545459681,
  "time_ms": 1545459681123,
  "channel": "futures.liquidates",
  "event": "subscribe",
  "result": {
    "status": "success"
  }
}
```

Liquidates notification

Spot & Margin

Perpetual Futures

Delivery Futures

Options

Unifi

Notify

- channel
 - futures.liquidates
- event
 - update
- params

field	type	description
result	Array	Array of objects

field	type	descript
entry_price	Float	Average entry pri
fill_price	Float	Average executio price
leverage	Float	Leverage
liq_price	Float	Liquidate price
margin	Float	Margin
mark_price	Float	Mark pri
order_id	Integer	Order id
order_price	Float	Commis: price
left	String/Integer	Order unfilled quantity

```
"channel": "futures.liquidates",
"event": "update",
"time": 1541505434,
"time_ms": 1541505434123,
"result": [
  {
    "entry_price": 209,
    "fill_price": 215.1,
    "left": "0",
    "leverage": 0.0,
    "liq_price": 213,
    "margin": 0.007816722941,
    "mark_price": 213,
    "order_id": 4093362,
    "order_price": 215.1,
    "size": "-124",
    "time": 1541486601,
    "time_ms": 1541486601123,
    "contract": "BTC_USD",
    "user": "1040xxxx"
  }
]
}
```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	size	String/Integer	Position original :		
	time	Integer	Time		
	time_ms	Integer	Time in millisecc		
	user	String	User id		
	contract	String	Futures contract name		

Cancel subscription

Unsubscribe liquidates update.

Request

- channel
 - futures.liquidates
- event
 - unsubscribe

Code example

```
import json, time
from websocket import create_connection

ws = create_connection("wss://fx-ws-1.gate.io")
req = {
    "time": int(time.time()),
    "channel": "futures.liquidates",
    "event": "unsubscribe",
    "payload": ["20011", "BTC_USD"],
    "auth": {
        "method": "api_key",
        "KEY": "xxxx",
        "SIGN": "xxxx"
    }
}
ws.send(json.dumps(req))
print(ws.recv())
```

The above command returns JSON structured like this:

```
time_ms": 1545459681123,
channel": "futures.liquidates",
event": "unsubscribe",
result": {
  status": "success"
}
}
```

Auto_deleverages API

Provides a way to receive user auto deleverages info.

Authentication required.

Auto_deleverages subscription

If you want to subscribe to auto leverage updates in all contracts, use `!all` in contract list.

Subscribe for user auto_deleverages update.

Request

- channel
 - futures.auto_deleverages
- event
 - subscribe
- params

Code example

```
import json, time
from websocket import create_connection

ws = create_connection("wss://fx-ws-...")

req = {
    "time": int(time.time()),
    "channel": "futures.auto_deleverages",
    "event": "subscribe",
    "payload": ["20011", "BTC_USD"],
    "auth": {
        "method": "api_key",
        "KEY": "xxxx",
        "SIGN": "xxxx"
    }
}
```

		Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	user id	String	yes	User ic		
	contract	String	yes	Future contra name		

The above command returns JSON structured like this:

```
json
{
  "time": 1545459681,
  "time_ms": 1545459681123,
  "channel": "futures.auto_deleverage",
  "event": "subscribe",
  "result": {
    "status": "success"
  }
}
```

Auto_deleverages notification

Notify auto_deleverages update.

Notify

- channel
 - futures.auto_deleverages
- event
 - update
- params

field	type	description
result	Array	Array of objects

field	type	descri
entry_price	Float	Entry p

```
json
{
  "channel": "futures.auto_deleverage",
  "event": "update",
  "time": 1541505434,
  "time_ms": 1541505434123,
  "result": [
    {
      "entry_price": 209,
      "fill_price": 215.1,
      "position_size": "10",
      "trade_size": "10",
      "time": 1541486601,
      "time_ms": 1541486601123,
      "contract": "BTC_USD",
      "user": "1040"
    }
  ]
}
```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
fill_price	Float	Execut price			
position_size	String/Integer	Positic size			
trade_size	String/Integer	Trade			
time	Integer	Time			
time_ms	Integer	Time i millise			
user	String	User ic			
contract	String	Future contra name			

Cancel subscription

Unsubscribe auto_deleverages update.

Code example

Request

- channel
 - futures.auto_deleverages
- event
 - unsubscribe

```
import json, time
from websocket import create_connection

ws = create_connection("wss://fx-ws-1.gate.io")
req = {
    "time": int(time.time()),
    "channel": "futures.auto_deleverages",
    "event": "unsubscribe",
    "payload": ["20011", "BTC_USD"],
    "auth": {
        "method": "api_key",
        "KEY": "xxxx",
        "SIGN": "xxxx"
    }
}
ws.send(json.dumps(req))
print(ws.recv())
```

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

```
json
{
  "time": 1545459681,
  "time_ms": 1545459681123,
  "channel": "futures.auto_deleverage",
  "event": "unsubscribe",
  "result": {
    "status": "success"
  }
}
```

Position_closes API

Provides a way to receive user position closes info.

Authentication required.

Position_closes subscription

If you want to subscribe to position close updates in all contracts, use `!all` in contract list.

Subscribe for user position_closes update.

Request

- channel

`futures.position_closes`

- event

`subscribe`

- params

Code example

```
python
import json, time
from websocket import create_connection

ws = create_connection("wss://fx-ws-1.gate.io")
req = {
  "time": int(time.time()),
  "channel": "futures.position_closes",
  "event": "subscribe",
  "payload": ["20011", "BTC_USD"],
  "auth": {
    "method": "api_key",
    "KEY": "xxxx",
    "SIGN": "xxxx"
  }
}
```

		Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	user id	String	yes	User ic		
	contract	String	yes	Future contra name		

The above command returns JSON structured like this:

json

```
{
  "time": 1545459681,
  "time_ms": 1545459681123,
  "channel": "futures.position_closes",
  "event": "subscribe",
  "result": {
    "status": "success"
  }
}
```

Position_closes notification

Notify position_closes update.

Notify

- channel
 - futures.position_closes
- event
 - update
- params

field	type	description
result	Array	Array of objects

json

```
{
  "channel": "futures.position_closes",
  "event": "update",
  "time": 1541505434,
  "time_ms": 1541505434123,
  "result": [
    {
      "contract": "BTC_USD",
      "pnl": -0.000624354791,
      "side": "long",
      "text": "web",
      "time": 1547198562,
      "time_ms": 1547198562123,
      "user": "211xxxx"
    }
  ]
}
```

		Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	contract	String	Futures contract name			
	pnl	Number	Profit & loss			
	side	String	Side (long or short)			
	text	String	Messages			
	time	Integer	Time			
	time_ms	Integer	Time in milliseconds			
	user	String	User id			

Cancel subscription

Unsubscribe position_closes update.

Code example

Request

- channel
 - futures.position_closes
- event
 - unsubscribe

```
import json, time
from websocket import create_connection

ws = create_connection("wss://fx-ws-1.gate.io")
req = {
    "time": int(time.time()),
    "channel": "futures.position_closes",
    "event": "unsubscribe",
    "payload": ["20011", "BTC_USD"],
    "auth": {
        "method": "api_key",
        "KEY": "xxxx",
        "SIGN": "xxxx"
    }
}
ws.send(json.dumps(req))
print(ws.recv())
```

json

```
{
  "time": 1545459681,
  "time_ms": 1545459681123,
  "channel": "futures.position_closes",
  "event": "unsubscribe",
  "result": {
    "status": "success"
  }
}
```

Balances API

Provides a way to receive user balances info.

Authentication required.

Balances subscription

Subscribe for user balances update.

Code example

Request

- channel
 - futures.balances
- event
 - subscribe
- params

parameter	type	required	description
user id	String	yes	User id

py

```
import json, time
from websocket import create_connection

ws = create_connection("wss://fx-ws-1.gate.io")

req = {
  "time": int(time.time()),
  "channel": "futures.balances",
  "event": "subscribe",
  "payload": ["20011"],
  "auth": {
    "method": "api_key",
    "KEY": "xxxx",
    "SIGN": "xxxx"
  }
}
```

The above command returns JSON structured like this:

json

```
{
  "time": 1545459681,
  "time_ms": 1545459681123,
  "channel": "futures.balances",
  "event": "subscribe",
  "result": {
    "status": "success"
  }
}
```

balances notification

Notify balances update.

Notify

- channel
 - futures.balances
- event
 - update
- params

field	type	description
result	Array	Array of objects

field	type	description
balance	Number	Balance after changed

json

```
{
  "channel": "futures.balances",
  "event": "update",
  "time": 1541505434,
  "time_ms": 1541505434123,
  "result": [
    {
      "balance": 9.998739899488,
      "change": -0.000002074115,
      "text": "BTC_USD:3914424",
      "time": 1547199246,
      "time_ms": 1547199246123,
      "type": "fee",
      "user": "211xxx",
      "currency": "btc"
    }
  ]
}
```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
change	Number	Change			
text	String	Messages			
time	Integer	Time			
time_ms	Integer	Time in milliseconds			
type	String	Type			
user	String	User id			
currency	String	Transfer unit			

Cancel subscription

Code example

```
py
import json, time
from websocket import create_connection

ws = create_connection("wss://fx-ws-1.gate.io")
req = {
    "time": int(time.time()),
    "channel": "futures.balances",
    "event": "unsubscribe",
    "payload": ["20011"],
    "auth": {
        "method": "api_key",
        "KEY": "xxxx",
        "SIGN": "xxxx"
    }
}
ws.send(json.dumps(req))
print(ws.recv())
```

The above command returns JSON structured like this:

Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
		<pre>time_ms: 1545459681123, channel: "futures.balances", event: "unsubscribe", result: { status: "success" } }</pre>		

Reduce_risk_limits API

Provides a way to receive user reduce risk limits info.

Authentication required.

Reduce_risk_limits subscription

If you want to subscribe to reduce risk limit updates in all contracts, use `!all` in contract list.

Subscribe for user `reduce_risk_limits` update.

Request

- channel
 - `futures.reduce_risk_limits`
- event
 - `subscribe`
- params

Code example

```
import json, time
from websocket import create_connection

ws = create_connection("wss://fx-ws-...")

req = {
    "time": int(time.time()),
    "channel": "futures.reduce_risk_...",
    "event": "subscribe",
    "payload": ["20011", "BTC_USD"],
    "auth": {
        "method": "api_key",
        "KEY": "xxxx",
        "SIGN": "xxxx"
    }
}
```

		Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	user id	String	yes	User ic		
	contract	String	yes	Future contra name		

The above command returns JSON structured like this:

json

```
{
  "time": 1545459681,
  "time_ms": 1545459681123,
  "channel": "futures.reduce_risk_lim
  "event": "subscribe",
  "result": {
    "status": "success"
  }
}
```

Reduce_risk_limits notification

Notify reduce risk limits update.

Notify

- channel
 - futures.reduce_risk_limits
- event
 - update
- params

field	type	description
result	Array	Array of objects

field	type	descriptio
cancel_orders	Integer	Cancel orders

json

```
{
  "time": 1551858330,
  "time_ms": 1551858330123,
  "channel": "futures.reduce_risk_lim
  "event": "update",
  "result": [
    {
      "cancel_orders": 0,
      "contract": "ETH_USD",
      "leverage_max": 10,
      "liq_price": 136.53,
      "maintenance_rate": 0.09,
      "risk_limit": 450,
      "time": 1551858330,
      "time_ms": 1551858330123,
      "user": "20011"
    }
  ]
}
```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	contract	String	Contract name		
	leverage_max	Integer	Leverage max		
	liq_price	Float	Liquidates price		
	maintenance_rate	Float	Maintenai rate		
	risk_limit	Integer	Risk limit		
	time	Integer	Time		
	time_ms	Integer	Time in millisecon		
	user	String	User id		

Cancel subscription

Unsubscribe reduce risk limits update.

Code example

Request

- channel

futures.reduce_risk_limits

- event

unsubscribe

```
import json, time
from websocket import create_connection

ws = create_connection("wss://fx-ws-1.gate.io")
req = {
    "time": int(time.time()),
    "channel": "futures.reduce_risk_limits",
    "event": "unsubscribe",
    "payload": ["20011", "BTC_USD"],
    "auth": {
        "method": "api_key",
        "KEY": "xxxx",
        "SIGN": "xxxx"
    }
}
```

Positions API

Provides a way to receive user positions info.

Authentication required.

Positions subscription

If you want to subscribe to position updates in all contracts, use `!all` in contract list.

Subscribe for user positions update.

Request

- channel
 - `futures.positions`
- event
 - `subscribe`
- params

parameter	type	required	descri
<code>user id</code>	String	yes	User ic field is deprec and is used a placeh
<code>contract</code>	String	yes	Future contra name

Code example

```
import json, time
from websocket import create_connection

ws = create_connection("wss://fx-ws-...")
req = {
    "time": int(time.time()),
    "channel": "futures.positions",
    "event": "subscribe",
    "payload": ["20011", "BTC_USD"],
    "auth": {
        "method": "api_key",
        "KEY": "xxxx",
        "SIGN": "xxxx"
    }
}
ws.send(json.dumps(req))
print(ws.recv())
```

The above command returns JSON structured like this:

```
{
  "time": 1545459681,
  "time_ms": 1545459681123,
  "channel": "futures.positions",
  "event": "subscribe",
```

}

Positions notification

Notify positions update.

Notify

- channel
 - futures.positions
- event
 - update
- params

field	type	description
result	Array	Array of objects

field	type
contract	String
cross_leverage_limit	Float
entry_price	Float
history_pnl	Float
history_point	Float

json

```
{
  "time": 1588212926,
  "time_ms": 1588212926123,
  "channel": "futures.positions",
  "event": "update",
  "result": [
    {
      "contract": "BTC_USD",
      "cross_leverage_limit": 0,
      "entry_price": 40000.366666611,
      "history_pnl": -0.000108569505,
      "history_point": 0,
      "last_close_pnl": -0.000050123,
      "leverage": 0,
      "leverage_max": 100,
      "liq_price": 0.1,
      "maintenance_rate": 0.005,
      "margin": 49.999890611186,
      "mode": "single",
      "realised_pnl": -1.25e-8,
      "realised_point": 0,
      "risk_limit": 100,
      "size": "3",
      "time": 1628736848,
      "time_ms": 1628736848321,
      "user": "110xxxxx",
      "update_id": 170919
    }
  ]
}
```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	last_close_pnl	Float			
	leverage	Integer			
	leverage_max	Integer			
	liq_price	Float			
	maintenance_rate	Float			
	margin	Float			
	realised_pnl	Float			
	realised_point	Float			
	risk_limit	Integer			
	size	String/Integer			
	time	Integer			

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	time_ms	Integer			
	user	String			
	update_id	Integer			

Cancel subscription

Unsubscribe positions update.

Code example

Request

- channel
 - futures.positions
- event
 - unsubscribe

py

```
import json, time
from websocket import create_connection

ws = create_connection("wss://fx-ws-4.gate.io")
req = {
    "time": int(time.time()),
    "channel": "futures.positions",
    "event": "unsubscribe",
    "payload": ["20011", "BTC_USD"],
    "auth": {
        "method": "api_key",
        "KEY": "xxxx",
        "SIGN": "xxxx"
    }
}
ws.send(json.dumps(req))
print(ws.recv())
```

The above command returns JSON structured like this:

json

```
{
    "time": 1545459681,
    "time_ms": 1545459681123,
    "channel": "futures.positions",
    "event": "unsubscribe",
```

}

Positions ADL API

The channel for pushing users' ADL ranking information

Authentication required.

Positions adl subscription

If you want to subscribe to adl updates in all contracts, use `!all` in contract list.

Subscribe for user positions adl update.

Request

- channel
 - `futures.position_adl_rank`
- event
 - `subscribe`
- params

parameter	type	required	descriptio
<code>contract</code>	String	yes	Futures contract name

Code example

```
import json, time
from websocket import create_connection

ws = create_connection("wss://ws-test")

req = {
    "time": int(time.time()),
    "channel": "futures.position_adl_rank",
    "event": "subscribe",
    "payload": ["BTC_USDT"],
    "auth": {
        "method": "api_key",
        "KEY": "xxxx",
        "SIGN": "xxxx"
    }
}

ws.send(json.dumps(req))
print(ws.recv())
```

The above command returns JSON structured like this:

```
{
  "time": 1545459681,
  "time_ms": 1545459681123,
```

Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
		<pre>"BTC_USDT"], "result": { "status": "success" } }</pre>		

Positions adl notification

Notify positions adl update.

Notify

- channel
 - futures.position_adl_rank
- event
 - update
- params

field	type	description
result	Array	Array of objects

field	type	description
contract	String	Futures contract name
mode	String	Position mode
rank_division	Integer	Ranking tier (1 ~ 6, with 1 being the highest priority for

json

```
{
  "time": 1588212926,
  "time_ms": 1588212926123,
  "channel": "futures.position_adl",
  "event": "update",
  "result": [
    {
      "contract": "BTC_USDT",
      "mode": "single",
      "rank_division": 1,
      "time_ms": 1588212926119,
      "user_id": 2124426495
    }
  ]
}
```

Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
------------------	----------------------	---------------------	---------	-------

		being the lowest priority for ADL 6: in liquidation, not included in ADL ranking)
time_ms	Integer	The millisecond timestamp of the message push
user_id	Integer	User Id

Cancel subscription

Unsubscribe positions adl update

Request

- channel

```
futures.position adl rank
```

- event

[unsubscribe](#)

Code example

```

import json, time
from websocket import create_connection

ws = create_connection("wss://ws-test")
req = {
    "time": int(time.time()),
    "channel": "futures.position_adl",
    "event": "unsubscribe",
    "payload": ["BTC_USDT"],
    "auth": {
        "method": "api_key",
        "KEY": "xxxx",
        "SIGN": "xxxx"
    }
}

ws.send(json.dumps(req))

print(ws.recv())

```

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

```
json
{
  "time": 1545459681,
  "time_ms": 1545459681123,
  "channel": "futures.position_adl_r",
  "event": "unsubscribe",
  "result": {
    "status": "success"
  }
}
```

Auto orders API

Provides a way to receive user auto orders info.

Authentication required.

Auto orders subscription

If you want to subscribe to auto order updates in all contracts, use `!all` in contract list.

Subscribe for user auto orders update.

Request

- channel
 - `futures.autoorders`
- event
 - `subscribe`
- params

Code example

```
py
import json, time
from websocket import create_connection

ws = create_connection("wss://fx-ws-1.gate.io")

req = {
    "time": int(time.time()),
    "channel": "futures.autoorders",
    "event": "subscribe",
    "payload": ["BTC_USDT"],
    "auth": {
        "method": "api_key",
        "KEY": "xxxx",
        "SIGN": "xxxx"
    }
}
```

Spot & Margin		Perpetual Futures	Delivery Futures	Options	Unifi
contract	String	yes	Future contract name	The above command returns JSON structured like this:	

json

```
{
  "time": 1545459681,
  "time_ms": 1545459681123,
  "channel": "futures.autoorders",
  "event": "subscribe",
  "result": {
    "status": "success"
  }
}
```

Auto orders notification

Notify auto orders update.

Notify

- channel
 - futures.autoorders
- event
 - update
- params

field	type	description
result	Array	Array of objects

field	type	description
user	Number	User id
trigger	Object	

json

```
{
  "time": 1596798126,
  "time_ms": 1596798126123,
  "channel": "futures.autoorders",
  "event": "update",
  "result": [
    {
      "user": 123456,
      "trigger": {
        "strategy_type": 0,
        "price_type": 0,
        "price": "10000",
        "rule": 2,
        "expiration": 86400
      },
      "initial": {
        "contract": "BTC_USDT",
        "size": "10",
        "price": "10000",
        "tif": "gtc",
        "text": "web",
        "iceberg": "0",
        "is_close": false,
        "is_reduce_only": false,
        "auto_size": ""
      }
    }
  ],
}
```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	initial	Object		<pre>"reason": "", "create_time": 1596798126, "name": "price_autoorders", "is_stop_order": false, "stop_trigger": { "rule": 0, "trigger_price": "", "order_price": "" }, "order_type": "close-long-order", "me_order_id": "213867453823" }]</pre>	
	id	Number	Auto order id		
	trade_id	Number	Trade id		
	status	String	Order status		
	reason	String	Change reason		
	create_time	Number	Create time		
	name	String	Name		
	is_stop_order	boolean	Is stop		
	stop_trigger	Object			
	order_type	String	Take-profit/stop-loss types, detail to http api		
	me_order_id	Number	Correspond order ID of order take-profit/stop-loss.		

Cancel subscription

Unsubscribe auto orders update.

Request

- channel
 - futures.autoorders

Code example

```
import json, time
from websocket import create_connection

ws = create_connection("wss://fx-ws-1.gate.io")
req = {
    "time": int(time.time()),
}
```

```
"auth": {
  "method": "api_key",
  "KEY": "xxxx",
  "SIGN": "xxxx"
}}
ws.send(json.dumps(req))
print(ws.recv())
```

The above command returns JSON structured like this:

json

```
{
  "time": 1545459681,
  "time_ms": 1545459681123,
  "channel": "futures.autoorders",
  "event": "unsubscribe",
  "result": {
    "status": "success"
  }
}
```

Futures Account Trade

Websocket API

WebSocket API allows placing , canceling , amending , querying orders through a WebSocket connection.

Websocket API Client Api Request

api requests initiated from the client follow a common JSON format, which contains the following fields:

Code example

py

```
#!/usr/bin/python
```

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

				Reque in secc Gap be reques and se time n exceec secon	<pre> import hashlib import websocket import threading API_KEY = "xxxxxx" SECRET = "xxxxxx" WS_URL = "wss://fx-ws.gateio.ws/v4/ws" CHANNEL_LOGIN = "futures.login" CHANNEL_ORDER_PLACE = "futures.order" def get_ts(): return int(time.time()) def get_ts_ms(): return int(time.time() * 1000) def get_signature(secret, channel, request): key = f"api\n{channel}\n{request}" return hmac.new(secret.encode(), key.encode(), hashlib.sha256).hexdigest() def build_login_request(): ts = get_ts() req_id = f"{get_ts_ms()}-1" request_param = b"" sign = get_signature(SECRET, CHANNEL_LOGIN, request_param) payload = { "api_key": API_KEY, "signature": sign, "timestamp": str(ts), "req_id": req_id } return { "time": ts, "channel": CHANNEL_LOGIN, "event": "api", "payload": payload } def build_order_request(): ts = get_ts() req_id = f"{get_ts_ms()}-2" order_param = { "contract": "BTC_USDT", "size": 6024, "iceberg": 0, </pre>
time	Integer	Yes			
id	Integer	No		Optio reques which sent b the ser help y identif reques server respor	
channel	String	Yes		WebSc chann subscr	
event	String	Yes		Fixed a	
payload	Object	Yes		Optio reques param	
» req_id	String	Yes		Uniqu identif the me Provid client.I return respor messa identif corres reques	
» timestamp	String	Yes		Signat in secc	

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

» <code>api_key</code>	String	Yes	Gate API user key
» <code>signature</code>	String	Yes	Authentication signature generated using Gate API secret key. See [Web API Authentication] (#Web API Authentication) section for details.
» <code>req_param</code>	[]Byte	Yes	Request parameter

Note that the type of `payload.req_param` is channel specific, Take `futures.order_place` for example, `payload.req_param` same as `apiv4 /futures/{settle}/orders` . You can place a limit order for BTC_USDT with example.

```

"stp_act": "-"
}

payload = {
    "req_id": req_id,
    "req_param": order_param
}

return {
    "time": ts,
    "channel": CHANNEL_ORDER_PLACE,
    "event": "api",
    "payload": payload
}

def on_message(ws, message):
    print(f"recv: {message}")

def on_error(ws, error):
    print(f"error: {error}")

def on_close(ws, close_status_code, close_msg):
    print("connection closed")

def on_open(ws):
    print("WebSocket opened")

login_payload = build_login_request_payload()
print("login payload:", login_payload)
ws.send(json.dumps(login_payload))

def delayed_order():
    time.sleep(2)
    order_payload = build_order_request_payload()
    print("order payload:", order_payload)
    ws.send(json.dumps(order_payload))

threading.Thread(target=delayed_order).start()

if __name__ == "__main__":
    ws = websocket.WebSocketApp(
        WS_URL,
        on_message=on_message,
        on_error=on_error,
        on_close=on_close,
        on_open=on_open
    )

```

Client request example

json

```
{
  "time": 1680772890,
  "channel": "futures.order_place",
  "event": "api",
  "payload": {
    "req_id": "xxxx",
    "req_param": {
      "contract": "BTC_USDT",
      "size": 10,
      "price": "80048.240000",
      "tif": "gtc",
      "text": "t-my-custom-id"
    }
  }
}
```

Websocket API Server Response

Server response includes ack response to client requests and api result callback message updates. Server responses follow a common JSON format, which contains the following fields:

Field	Type
request_id	String
header	Map
» response_time	String
» channel	String
» event	String
» client_id	String

Server ack response example

json

```
{
  "request_id": "request-id-1",
  "ack": true,
  "header": {
    "response_time": "1681195121499",
    "status": "200",
    "channel": "futures.order_place",
    "event": "api",
    "client_id": "::1-0x140031563c0",
    "x_in_time": 1681985856667508,
    "x_out_time": 1681985856667598,
    "conn_id": "5e74253e9c793974",
    "conn_trace_id": "1bde5aaa0acf2f5",
    "trace_id": "e410abb5f74b4afc519e",
    "x_gate_ratelimit_requests_remain": 100,
    "x_gate_ratelimit_limit": 100,
    "x_gate_ratelimit_reset_timestamp": 1681195121499
  },
  "data": {
    "result": {
      "req_id": "request-id-1",

```

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

» x_in_time

Integer

» x_out_time

Integer

» x_gate_ratelimit_requests_remain

Integer

» x_gate_ratelimit_limit

Integer

» x_gat_ratelimit_reset_timestamp

Integer

» conn_id

String

» conn_trace_id

String

» trace_id

String

data

Object

» result

Object

```

"price": "31503.280000",
"tif": "gtc",
"text": "t-my-custom-id"
}
}
}
}

```

Server api response example

```

json
{
  "request_id": "request-id-1",
  "ack": false,
  "header": {
    "response_time": "1681195121639",
    "status": "200",
    "channel": "futures.order_place",
    "event": "api",
    "client_id": "::1-0x140031563c0",
    "x_in_time": 1681985856667508,
    "x_out_time": 1681985856667598,
    "conn_id": "5e74253e9c793974",
    "conn_trace_id": "1bde5aaa0acf2f5",
    "trace_id": "e410abb5f74b4afc519e",
    "x_gate_ratelimit_requests_remain": 100,
    "x_gate_ratelimit_limit": 100,
    "x_gate_ratelimit_reset_timestamp": 1681195121639
  },
  "data": {
    "result": {
      "id": 74046511,
      "user": 6790020,
      "create_time": 1681195121.754,
      "finish_time": 1681195121.754,
      "finish_as": "filled",
      "status": "finished",
      "contract": "BTC_USDT",
      "size": "10",
      "price": "31503.3",
      "tif": "gtc",
      "fill_price": "31500",
      "text": "t-my-custom-id",
      "tkfr": "0.0003",
      "mkfr": "0",
      "stp_id": 2,
      "stp_act": "cn",
      "amend_text": "-"
    }
  }
}

```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	» errs	Object			
	»» label	String			
	»» message	String			

Error

Error object has the following format:

Field	Type	Description
label	String	Denotes error type in string format
message	String	Detailed error message

Explanation of rate limit-related error codes:

Code	Description
100	Internal rate limiting exception error
311	Futures rate limit
312	Futures rate limiting based on fill rate

Error Response Notification example

```

{
  "request_id": "request-id-1",
  "ack": false,
  "header": {
    "response_time": "1681195360034",
    "status": "401",
    "channel": "futures.login",
    "event": "api",
    "client_id": "::1-0x140001a2600",
    "x_in_time": 1681985856667508,
    "x_out_time": 1681985856667598,
    "conn_id": "5e74253e9c793974",
    "conn_trace_id": "1bde5aaa0acf2f",
    "trace_id": "e410abb5f74b4afc519e"
  },
  "data": {
    "errs": {
      "label": "INVALID_KEY",
      "message": "Invalid key provided"
    }
  }
}

```

Error Response Notification example (rate limit)

Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
------------------	----------------------	---------------------	---------	-------

```
"header": {
  "response_time": "1677816784084",
  "status": "429",
  "channel": "futures.order_place",
  "event": "api",
  "client_id": "::-1-0x14002ba2300",
  "x_in_time": 1681985856667508,
  "x_out_time": 1681985856667598,
  "conn_id": "5e74253e9c793974",
  "conn_trace_id": "1bde5aaa0ac2f5",
  "trace_id": "e410abb5f74b4afc519e",
  "x_gate_ratelimit_limit": 100,
  "x_gate_ratelimit_reset_timestamp": 1681985856667508
},
"data": {
  "errs": {
    "label": "TOO_MANY_REQUESTS",
    "message": "Request Rate limit"
  }
}
```

Login

Note: the GateAPIv4 key pair you used MUST have future Corresponding permissions(eg: order-place channel must have future write permissions), and your outbound IP address must be in the key's IP whitelist if its whitelist is enabled.

Login Request

Client Api Request

Payload format:

Code example

```
import hmac
import hashlib
import json
```

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

				Request which sent to the server help identify which request server response it's different from outside
req_id	string	Yes		
api_key	string	Yes		Apiv4
headers	object	Yes		Apiv4 custom headers
signature	string	Yes		Apiv4 signature
timestamp	string	Yes		Unix timestamp second

WebSocket api operation authentication uses the same signature calculation method with Gate APIv4 API, i.e., `HexEncode(HMAC_SHA512(secret, signature_string))`, but has the following differences:

- Signature string concatenation method: "`<event>\n<channel>\n<req_param>\n<timestamp>`" where `<event>`, `<channel>`, `<req_param>`, `<timestamp>` are corresponding request information
- `req_param` in `login` channel always empty string
- Authentication information are sent in request body in field `payload`.

```
def get_api_signature(secret, channel, event, req_param, timestamp):
    key = f"api\n{channel}\n{request_param}\n{timestamp}"
    hash_object = hmac.new(secret.encode(), key.encode(), hashlib.sha512)
    return hash_object.hexdigest()

class ApiPayload:
    def __init__(self, api_key, signature, timestamp, req_id, request_param):
        self.api_key = api_key
        self.signature = signature
        self.timestamp = timestamp
        self.req_id = req_id
        self.request_param = request_param

class ApiRequest:
    def __init__(self, ts, channel, event, req_param, payload):
        self.time = ts
        self.channel = channel
        self.event = event
        self.payload = payload

def main():
    api_key = "YOUR_API_KEY"
    secret = "YOUR_API_SECRET"
    request_param = ""
    channel = "futures.login"
    ts = int(time.time())
    request_id = f"{int(time.time())}"

    payload = ApiPayload(
        api_key=api_key,
        signature=get_api_signature(secret, channel, event, req_param, timestamp),
        timestamp=str(ts),
        req_id=request_id,
        request_param=request_param
    )

    req = ApiRequest(ts=ts, channel=channel, event=event, req_param=req_param, payload=payload)

    print(get_api_signature(secret, channel, event, req_param, timestamp))

    req_json = json.dumps(req, default=lambda o: o.__dict__)
    print(req_json)

    # Connect to WebSocket
    ws_url = "wss://fx-ws.gateio.ws/v4"
    websocket.enableTrace(False)
    ws = websocket.create_connection(ws_url)

    # Function to receive messages
```

Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
		<pre>message = ws.recv() print(f"recv: {message}") except Exception as e: print(f"Error receiving message: {e}") ws.close() break # Start receiving messages in a separate thread import threading receive_thread = threading.Thread(target=receive_message) receive_thread.start() # Send the request ws.send(req_json) # Keep the main thread running receive_thread.join() if __name__ == "__main__": main()</pre>		

Request example

```
json
{
  "time": 1681984544,
  "channel": "futures.login",
  "event": "api",
  "payload": {
    "api_key": "ea83fad2604399da16bf9...",
    "signature": "6fa3824c8141f2b228...",
    "timestamp": "1681984544",
    "req_id": "request-1"
  }
}
```

Login Notification

Result format:

Field	Type	Description
request_id	String	Unique identifier of

Login Response example

```
json
{
  "request_id": "request-1",
  "header": {
    "response_time": "1681985856666"
  }
}
```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
			the message		
header	Map	Response meta info	<pre>"clientId": "", "x_in_time": 1681985856667508, "x_out_time": 1681985856667598, "conn_id": "5e74253e9c793974", "conn_trace_id": "1bde5aaa0acf2f5", "trace_id": "e410abb5f74b4afc519e", }, "data": { "result": { "api_key": "ea83fad2604399da16", "uid": "110284739" } }</pre>		
» response_time	String	Response send time in mill			
» channel	String	Request channel			
» event	String	Request event			
» client_id	String	Unique client id			
» x_in_time	Integer	time to receive the request (in microseconds)			
» x_out_time	Integer	time to return response (in microseconds)			
» conn_id	String	Connection ID established with the client (remains consistent for the same connection)			
» conn_trace_id	String	TraceId to establish connection with client			
» trace_id	String	TraceId for executing order operation			



Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

<code>data</code>	Object	Response data of the request
<code>» result</code>	Object	If this is ack response, result is the payload of the request, otherwise result is the response of the api
<code>»» api_key</code>	String	Login success apikey
<code>»» uid</code>	String	Login user id
<code>» errs</code>	Object	It is only available when the request fails
<code>»» label</code>	String	Denotes error type in string format
<code>»» message</code>	String	Detailed error message

Order Place

`futures.order_place`

You can place orders with this channel and event `api` .

function as api below:

Order Place Request

Payload format:

Field	Type	Required	Desc
req_id	string	Yes	Request ID which sent the server to help identify the request and its response from outside
req_param	object	Yes	API model byte data of the API order model 
req_header	object	No	API custom header

req_param JSON byte data of the API order model:

Field	Type	Required
contract	string	true

Code example: Login is required before making requests

py

```
import time
import json

# pip install websocket_client
from websocket import create_connection

api_order = {
    "contract": "BTC_USDT",
    "size": 10,
    "price": "31503.280000",
    "tif": "gtc",
    "text": "t-my-custom-id"
}

ws = create_connection("wss://fx-ws.gate.io")
ws.send(json.dumps(
    {
        "time": int(time.time()),
        "channel": "futures.order_place",
        "event": "api",
        "payload": {
            "req_id": "1ewq-3123w-5",
            "req_param": api_order
        }
    }
))

print(ws.recv())
```

Request example

json

```
{
  "time": 1681195484,
  "channel": "futures.order_place",
  "event": "api",
  "payload": {
    "req_id": "request-id-1",
    "req_param": {
      "contract": "BTC_USDT",
```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	size	int64	true	"text": "t-my-custom-id" } } }	
	iceberg	int64	true		
	price	string	false		
	close	bool	false		
	reduce_only	bool	false		
	tif	string	false		
	text	string	false		
	auto_size	string	false		

req_header Custom header data:

Field	Type	Required	Description
x-gate-exptime	string	false	Specifies expiration timestamp (in

Detail

- gtc: Good Till Canceled
- ioc: Immediate Or Cancelled, accepted only
- poc: Pending Or Cancelled, for posted orders only, always incurs maker fees
- fok: Fill Or Kill, must be completely filled or not filled at all

- Must start with "t-"
- Excluding "t-", the length cannot exceed 28 bytes
- The input can only contain numbers, letters, underscores (_), hyphens (-), or dots (.)
- If left empty, the default is apiv4-ws, from WebSocket
- web: From the web
- api: From the API
- app: From the mobile app
- auto_deleveraging: From ADL
- liquidation: From liquidation
- insurance: From insurance

After the user joins the **STP Group** , they can use **stp act** to limit their self-trade

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

`p` , passing the parameter will return an error with `stp_act` . If the user does not use `stp_act` when placing an order, `stp_act` will return `-` .

- `cn`: Cancel the latest order, cancel the new order and keep the old one
- `co`: Cancel the oldest order, cancel the old order and keep the new one
- `cb`: Cancel both, cancel both the new and old orders

Ack Notification

Ack Notification example for an order

```

{
  "request_id": "request-id-1",
  "ack": true,
  "header": {
    "response_time": "1681195484268",
    "status": "200",
    "channel": "futures.order_place",
    "event": "api",
    "client_id": "::1-0x140001a2600",
    "x_in_time": 1681985856667508,
    "x_out_time": 1681985856667598,
    "conn_trace_id": "1bde5aaa0acf2f5",
    "trace_id": "e410abb5f74b4afc519e",
    "conn_id": "5e74253e9c793974",
    "x_gate_ratelimit_requests_remain": 100,
    "x_gate_ratelimit_limit": 100,
    "x_gat_ratelimit_reset_timestamp": 1681985856667598
  },
  "data": {
    "result": {
      "req_id": "request-id-1",
      "req_header": null,
      "req_param": {
        "contract": "BTC_USDT",
        "size": "10",
        "price": "31503.280000",
        "tif": "gtc",
        "text": "t-my-custom-id"
      }
    }
  }
}

```

Order Place Notification

Return of order information for placing an order

Result format:

Field	Type
request_id	String
ack	Bool
header	Map
» response_time	String
» channel	String
» event	String

Response Notification example

json

```
{
  "request_id": "request-id-1",
  "ack": false,
  "header": {
    "response_time": "1681195484360",
    "status": "200",
    "channel": "futures.order_place",
    "event": "api",
    "client_id": "::1-0x140001a2600",
    "x_in_time": 1681985856667508,
    "x_out_time": 1681985856667598,
    "conn_id": "5e74253e9c793974",
    "conn_trace_id": "1bde5aaa0acf2f5",
    "trace_id": "e410abb5f74b4afc519e",
    "x_gate_ratelimit_requests_remain": 100,
    "x_gate_ratelimit_limit": 100,
    "x_gat_ratelimit_reset_timestamp": 1681195484360
  },
  "data": {
    "result": {
      "id": 74046514,
      "user": 6790020,
      "create_time": 1681195484.462,
      "finish_time": 1681195484.462,
      "finish_as": "filled",
      "status": "finished",
      "contract": "BTC_USDT",
      "size": "10",
      "price": "31503.3",
      "tif": "gtc",
      "fill_price": "31500",
      "text": "t-my-custom-id",
      "tkfr": "0.0003",
      "mkfr": "0",
      "stp_id": 2,
      "stp_act": "cn",
      "amend_text": "-"
    }
  }
}
```

Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
» client_id	String			
» x_in_time	Integer			
» x_out_time	Integer			
» conn_trace_id	String			
» trace_id	String			
» x_gate_ratelimit_requests_remain	Integer			
» x_gate_ratelimit_limit	Integer			
» x_gat_ratelimit_reset_timestamp	Integer			



Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
» conn_id		String		
data		Object		
» result		Object		
» errs		Object		
»» label		String		
»» message		String		

Order Batch Place

```
futures.order_batch_place
```

You can batch place orders with this channel and event `api` .

function as api below:

json

POST /futures/{settle}/batch_orders

Order Batch Place Request

Payload format:

Field	Type	Required	Desc
req_id	string	Yes	Request ID which sent to the server to help identify the request. It's different from outside request ID.
req_param	object	Yes	API order model as a JSON byte array. Refer to the API order model to API order placement.
req_header	object	No	API custom header data.

req_param The JSON byte data for the API order model can refer to a single order placement, or an array of multiple single orders. For details, refer to the order placement documentation.

req_header Custom header data:

Code example: Login is required before making requests

```
import time
import json

# pip install websocket_client
from websocket import create_connection

api_order=[
    {
        "contract": "BTC_USDT",
        "size": 10,
        "price": "31403.180000",
        "tif": "gtc",
        "text": "t-my-custom-id"
    }
]

ws = create_connection("wss://fx-ws.gate.io")
ws.send(json.dumps({
    "time": int(time.time()),
    "channel": "futures.order_batch_place",
    "event": "api",
    "payload": {
        "header":{
            "x-gate-channel-id":"xxxx"
        },
        "req_id": "1ewq-3123w-5",
        "req_param": api_order
    }
}))

print(ws.recv())
```

Request example

```
{
  "time": 1681196536,
  "channel": "futures.order_batch_place",
  "event": "api",
  "payload": {
    "req_id": "request-id-6",
  }
}
```

		Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	x-gate-exptime	string	false	Specifies expiration timestamp (in milliseconds). If the request time received the WebSocket later than expiration time, the request will be rejected.	<pre>"size": "10", "price": "31403.180000", "tif": "gtc", "text": "t-my-custom-id" }] }</pre>	

Order Batch Place Ack Notification

Ack Notification example

```
json
{
  "request_id": "request-id-6",
  "ack": true,
  "header": {
    "response_time": "1681196536283",
    "status": "200",
    "channel": "futures.order_batch",
    "event": "api",
    "client_id": "::1-0x14002cfa0c0",
    "x_in_time": 1681985856667508,
    "x_out_time": 1681985856667598,
    "conn_id": "5e74253e9c793974",
    "conn_trace_id": "1bde5aaa0acf2f",
    "trace_id": "e410abb5f74b4afc519",
    "x_gate_ratelimit_requests_remain": 99,
    "x_gate_ratelimit_limit": 100,
    "x_gate_ratelimit_reset_timestamp": 1681985856667598
  },
  "data": {
    "result": {
      "req_id": "request-id-6",
      "req_header": null,
      "req_param": [

```

Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
				<pre>"price": "31403.180000", "tif": "gtc", "text": "t-my-custom-id" }] } } }</pre>

Order Batch Place Notification

Return of order information for batch placing order

Result format:

Field	Type
request_id	String
ack	Bool
header	Map

Response Notification example

json

```
{
  "request_id": "request-id-6",
  "ack": false,
  "header": {
    "response_time": "1681196536532",
    "status": "200",
    "channel": "futures.order_batch",
    "event": "api",
    "client_id": "::1-0x14002cfa0c0",
    "x_in_time": 1681985856667508,
    "x_out_time": 1681985856667598,
    "conn_id": "5e74253e9c793974",
    "conn_trace_id": "1bde5aaa0acf2f",
    "trace_id": "e410abb5f74b4afc519",
    "x_gate_ratelimit_requests_remain": 99,
    "x_gate_ratelimit_limit": 100,
    "x_gat_ratelimit_reset_timestamp": 1681196536532
  },
  "data": {
    "result": [
      {
        "succeeded": true,
        "id": 74046545,
        "user": 6790020,
        "create_time": 1681196536.59,
        "status": "open",
        "contract": "BTC_USDT",
        "size": "10",
        "price": "31403.2",
        "tif": "gtc",
        "left": "10",
        "fill_price": "0",
        "text": "t-my-custom-id",
```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
» <code>response_time</code>		String	<pre>] }</pre>		
» <code>channel</code>		String			
» <code>event</code>		String			
» <code>client_id</code>		String			
» <code>x_in_time</code>		Integer			
» <code>x_out_time</code>		Integer			
» <code>conn_trace_id</code>		String			
» <code>trace_id</code>		String			
» <code>x_gate_ratelimit_requests_remain</code>		Integer			
» <code>x_gate_ratelimit_limit</code>		Integer			
» <code>x_gat_ratelimit_reset_timestamp</code>		Integer			

Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
» conn_id	String			
data	Object			
» result	Object			
» errs	Object			
»» label	String			
»» message	String			

Order Cancel

```
futures.order_cancel
```

You can cancel an open order with this channel and event `api` .

function as api below:



Order Cancel Request

Payload format:

Field	Type	Required	Desc
req_id	string	Yes	Request ID which sent the request to help identify the request which server responds to it's data from outside
req_param	object	Yes	API order model JSON data to API
req_header	object	No	API header

req_param API order model JSON data:

Field	Type	Required	Description
order_id	string	Yes	Order ID returned when the order was successfully created by user specific custom

Code example: Login is required before making requests

py

```
import time
import json

# pip install websocket_client
from websocket import create_connection

ws = create_connection("wss://fx-ws.gate.io")

api_cancel_order = {
    "order_id": "74046514"
}

ws.send(json.dumps({
    "time": int(time.time()),
    "channel": "futures.order_cancel",
    "event": "api",
    "payload": {
        "req_id": "1ewq-3123w-5",
        "req_param": api_cancel_order
    }
}))

print(ws.recv())
```

Order cancel request example

json

```
{
  "time": 1681195485,
  "channel": "futures.order_cancel",
  "event": "api",
  "payload": {
    "req_id": "request-id-5",
    "req_param": {
      "order_id": "74046514"
    }
  }
}
```

Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi

req_header Custom header data:

Field	Type	Required	Description
x-gate-exptime	string	false	Specifies expiration timestamp (in milliseconds). If the request time received the WebSocket later than expiration time, the request will be rejected.

Order Status Notification

Result format:

Field	Type
request_id	String
header	Map
» response_time	String
» channel	String

Order status notification example

```
json
{
  "request_id": "request-id-5",
  "header": {
    "response_time": "1681196536282",
    "status": "200",
    "channel": "futures.order_cancel",
    "event": "api",
    "client_id": "::1-0x14002cfa0c0",
    "x_in_time": 1681985856667508,
    "x_out_time": 1681985856667598,
    "conn_id": "5e74253e9c793974",
    "conn_trace_id": "1bde5aaa0acf2f",
    "trace_id": "e410abb5f74b4afc519e",
    "x_gate_ratelimit_requests_remain": 100,
    "x_gate_ratelimit_limit": 100,
    "x_gate_ratelimit_reset_timestamp": 1681196536282
  },
}
```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
	» event	String	<pre>"user": 6790020, "create_time": 1681196535.01, "finish_time": 1681196536.343, "finish_as": "cancelled", "status": "finished", "contract": "BTC_USDT", "size": "10", "price": "31303.2", "tif": "gtc", "left": "10", "fill_price": "0", "text": "t-my-custom-id", "tkfr": "0.0003", "mkfr": "0", "stp_id": 2, "stp_act": "cn", "amend_text": "-" } } }</pre>		
	» client_id	String			
	» x_in_time	Integer			
	» x_out_time	Integer			
	» conn_trace_id	String			
	» trace_id	String			
	» x_gate_ratelimit_requests_remain	Integer			
	» x_gate_ratelimit_limit	Integer			
	» x_gat_ratelimit_reset_timestamp	Integer			

» conn_id	String
data	Object
» result	Object
» errs	Object
»» label	String
»» message	String

Multiple distinct order IDs can be specified. Each request can cancel a maximum of 20 records.

function as api below:

```
json
POST /futures/{settle}/batch_cancel_order
```

Order Cancel All With Id List Request

Payload format:

Field	Type	Required	Description
req_id	string	Yes	request id which sent to the server help identify the request which server response it's data from outside
req_param	array	Yes	Order
req_header	object	No	api's header

req_header Custom header data:

Field	Type	Required	Description
x-gate-expirtime	string	false	Specifies expiration timestamp

Code example: Login is required before making requests

```
py
#!/usr/bin/python

import time
import json
# pip install websocket_client
from websocket import create_connection

ws = create_connection("wss://fx-ws.gate.io")
cancelWithIdsParam = ["1694883366", "1694883367"]
ws.send(json.dumps({
    "time":int(time.time()),
    "channel":"futures.order_cancel_order",
    "event":"api",
    "payload":{
        "req_id":"test_1",
        "req_param": cancelWithIdsParam
    }
}))

print(ws.recv())
```

Client request example

```
json
{
  "time": 1681986208,
  "channel": "futures.order_cancel_order",
  "event": "api",
```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unified
100 milliseconds If the request is received within the WebSocket connection's expiration time, the request will be rejected.			<pre> { "id": "1700664343", "type": "123" } </pre>		

Order Cancel All With Id List Notification

Result format:

Field	Type	Description
<code>request_id</code>	String	Unique identifier of the message
<code>header</code>	Map	response meta info
» <code>response_time</code>	String	response send time in mill
» <code>channel</code>	String	request channel
» <code>event</code>	String	request event
» <code>client_id</code>	String	Unique client id
» <code>x_in_time</code>	Integer	time to receive the request (in microseconds)

Order cancel notification example

```

{
  "request_id": "request-9",
  "header": {
    "response_time": "1681986208564",
    "status": "200",
    "channel": "futures.order_cancel",
    "event": "api",
    "client_id": "::-1-0x140001623c0",
    "x_in_time": 1681985856667508,
    "x_out_time": 1681985856667598,
    "conn_id": "5e74253e9c793974",
    "conn_trace_id": "1bde5aaa0acf2f",
    "trace_id": "e410abb5f74b4afc519e"
  },
  "data": {
    "result": [
      {
        "id": "1694883366",
        "user_id": 111,
        "succeeded": true
      },
      {
        "id": "123",
        "user_id": 111,
        "message": "ORDER_NOT_FOUND"
      }
    ]
  }
}

```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
» <code>x_out_time</code>	Integer	time to return response (in microseconds)			
» <code>conn_id</code>	String	Connection ID established with the client (remains consistent for the same connection)			
» <code>conn_trace_id</code>	String	TraceId to establish connection with client			
» <code>trace_id</code>	String	TraceId for executing order operation			
<code>data</code>	Object	Response data of the request			
» <code>result</code>	Object	response detail to api			
» <code>errs</code>	Object	It is only available when the request fails			
»» <code>label</code>	String	denotes error type in string format			
»» <code>message</code>	String	detailed error message			

Cancel all open orders matched

futures.order_cancel_cp

You can cancel all open orders with this channel and event `api` .

function as api below:

```
DELETE /futures/{settle}/orders
```

json

Cancel all open orders matched Request

Payload format:

Field	Type	Required	Desc
req_id	string	Yes	Request ID which sent the request to help identify which request server responds it's data from outside
req_param	object	Yes	Data ↗
req_header	object	No	API header

req_header Custom header data:

Code example: Login is required before making requests

```
import time
import json

# pip install websocket_client
from websocket import create_connection

ws = create_connection("wss://fx-ws.gate.io")

api_cancel_all_order = {
    "contract": "BTC_USDT",
    "side": "bid"
}

ws.send(json.dumps({
    "time": int(time.time()),
    "channel": "futures.order_cancel_cp",
    "event": "api",
    "payload": {
        "req_id": "1ewq-3123w-5",
        "req_param": api_cancel_all_order
    }
}))

print(ws.recv())
```

py

Spot & Margin			Perpetual Futures	Delivery Futures	Options	Unifi
x-gate-exptime	string	false	Specifies expiration timestamp (in milliseconds). If the request time received the WebSocket later than expiration time, the request will be rejected.	<pre>{ "time": 1681196537, "channel": "futures.order_cancel", "event": "api", "payload": { "req_id": "request-id-7", "req_param": { "contract": "BTC_USDT", "side": "bid" } } }</pre>		

Cancel all open orders matched
Notification

Result format:

Field	Type
request_id	String
header	Map
» response_time	String
» channel	String
» event	String
» client_id	String

Order cancel notification example

```
json
{
  "request_id": "request-id-7",
  "header": {
    "response_time": "1681196537567",
    "status": "200",
    "channel": "futures.order_cancel",
    "event": "api",
    "client_id": "::1-0x14002cfa0c0",
    "x_in_time": 1681985856667508,
    "x_out_time": 1681985856667598,
    "conn_id": "5e74253e9c793974",
    "conn_trace_id": "1bde5aaa0acf2f5",
    "trace_id": "e410abb5f74b4afc519e",
    "x_gate_ratelimit_requests_remain": 100,
    "x_gate_ratelimit_limit": 100,
    "x_gate_ratelimit_reset_timestamp": 1681196537567
  },
  "data": {
    "result": [
      {
        "id": 74046545,
        "user": 6790020,
        "create_time": 1681196536.592
      }
    ]
  }
}
```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
			<pre>"contract": "BTC_USDT", "size": "10", "price": "31403.2", "tif": "gtc", "left": "10", "fill_price": "0", "text": "t-my-custom-id", "tkfr": "0.0003", "mkfr": "0", "stp_id": 2, "stp_act": "cn", "amend_text": "- " }] }</pre>		
» x_in_time		Integer			
» x_out_time		Integer			
» conn_trace_id		String			
» trace_id		String			
» x_gate_ratelimit_requests_remain		Integer			
» x_gate_ratelimit_limit		Integer			
» x_gat_ratelimit_reset_timestamp		Integer			



Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
» <code>conn_id</code>	String			
<code>data</code>	Object			
» <code>result</code>	Object			
» <code>errs</code>	Object			
»» <code>label</code>	String			
»» <code>message</code>	String			

Order Amend

`futures.order_amend`

You can amend an open order with this channel and event `api` .

function as api below:



Order Amend Request

Payload format:

Field	Type	Required	Description
req_id	string	Yes	Request ID which is sent to the server to help identify the request. The server responds with the request ID from outside.
req_param	object	Yes	API order parameters to amend.
req_header	object	No	API header data.

req_header Custom header data:

Field	Type	Required	Description
x-gate-exptime	string	false	Specifies expiration timestamp (in milliseconds). If the request is received after the timestamp, the WebSocket connection will be closed.

Code example: Login is required before making requests

```
import time
import json

# pip install websocket_client
from websocket import create_connection

ws = create_connection("wss://fx-ws.gate.io")

api_amend_order = {
    "order_id": "74046543",
    "price": "31303.180000"
}

ws.send(json.dumps({
    "time": int(time.time()),
    "channel": "futures.order_amend",
    "event": "api",
    "payload": {
        "req_id": "1ewq-3123w-5",
        "req_param": api_amend_order
    }
}))

print(ws.recv())
```

Client request example

```
{
  "time": 1681196536,
  "channel": "futures.order_amend",
  "event": "api",
  "payload": {
    "req_id": "request-id-4",
    "req_param": {
      "order_id": "74046543",
      "price": "31303.180000"
    }
  }
}
```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
After the expiration time, the request will be rejected					

Detail descriptions

size: New order size, including filled part.

- If new size is less than or equal to filled size, the order will be cancelled.
- Order side must be identical to the original one.
- Close order size cannot be changed.
- For reduce only orders, increasing size may leads to other reduce only orders being cancelled.
- If price is not changed, decreasing size will not change its precedence in order book, while increasing will move it to the last at current price.



Order Amend Notification

Result format:

Field	Type
request_id	String
header	Map
» response_time	String
» channel	String

Order amend notification example

```
{
  "request_id": "request-id-4",
  "header": {
    "response_time": "1681196536251",
    "status": "200",
    "channel": "futures.order_amend",
    "event": "api",
    "client_id": ">::1-0x14002cfa0c0",
    "x_in_time": 1681985856667508,
    "x_out_time": 1681985856667598,
    "conn_id": "5e74253e9c793974",
    "conn_trace_id": "1bde5aaa0acf2f",
    "trace_id": "e410abb5f74b4afc519e",
    "x_gate_ratelimit_requests_remain": 100,
    "x_gate_ratelimit_limit": 100,
    "x_gat_ratelimit_reset_timestamp": 1681985856667508
  },
  "data": {
```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
» event		String	<pre>"create_time": 1681196535.01, "status": "open", "contract": "BTC_USDT", "size": "10", "price": "31303.2", "tif": "gtc", "left": "10", "fill_price": "0", "text": "t-my-custom-id", "tkfr": "0.0003", "mkfr": "0", "stp_id": 2, "stp_act": "cn", "amend_text": "-" } } }</pre>		
» client_id		String			
» x_in_time		Integer			
» x_out_time		Integer			
» conn_trace_id		String			
» trace_id		String			
» x_gate_ratelimit_requests_remain		Integer			
» x_gate_ratelimit_limit		Integer			
» x_gat_ratelimit_reset_timestamp		Integer			

» conn_id	String
data	Object
» result	Object
» errs	Object
»» label	String
»» message	String

```
futures.order_list
```

function as api below:

```
GET /futures/{settle}/orders
```

Order List Request

Payload format:

Field	Type	Required	Descr
req_id	string	Yes	Request ID which is sent to the server to help identify the request. It's different from the order ID.
req_param	object	Yes	API list details ↗

Code example: Login is required before making requests

```
import time
import json

# pip install websocket_client
from websocket import create_connection

ws = create_connection("wss://fx-ws.gate.io")

api_list_order = {
    "contract": "BTC_USDT",
    "status": "open"
}

ws.send(json.dumps({
    "time": int(time.time()),
    "channel": "futures.order_list",
    "event": "api",
    "payload": {
        "req_id": "1ewq-3123w-5",
        "req_param": api_list_order
    }
}))

print(ws.recv())
```

Client request example

```
{
  "time": 1681196535,
  "channel": "futures.order_list",
  "event": "api",
  "payload": {
    "req_id": "request-id-3",
    "req_param": {
```

Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi

Order List Notification

Result format:

Field	Type	Description
request_id	String	Unique identifier of the message
header	Map	Response meta info
» response_time	String	Response send time in mill
» channel	String	Request channel
» event	String	Request event
» client_id	String	Unique client id
» x_in_time	Integer	time to receive the request (in microseconds)
» x_out_time	Integer	time to return response (in microseconds)
» conn_id	String	Connection ID established with the client (remains consistent for

Order list notification example

```
json
{
  "request_id": "request-id-3",
  "header": {
    "response_time": "1681196536017",
    "status": "200",
    "channel": "futures.order_list",
    "event": "api",
    "client_id": "::1-0x14002cfa0c0",
    "x_in_time": 1681985856667508,
    "x_out_time": 1681985856667598,
    "conn_id": "5e74253e9c793974",
    "conn_trace_id": "1bde5aaa0acf2f",
    "trace_id": "e410abb5f74b4afc519"
  },
  "data": {
    "result": [
      {
        "id": 74046543,
        "user": 6790020,
        "create_time": 1681196535.01,
        "finish_time": 1681196535.01,
        "update_time": 1681196535.01,
        "finish_as": "filled",
        "status": "open",
        "contract": "BTC_USDT",
        "size": "10",
        "price": "31403.2",
        "tif": "gtc",
        "left": "10",
        "fill_price": "0",
        "text": "t-my-custom-id",
        "tkfr": "0.0003",
        "mkfr": "0",
        "stp_id": 2,
        "stp_act": "cn",
        "amend_text": "-"
      }
    ]
  }
}
```

	Spot & Margin	Perpetual Futures	Delivery Futures	Options	Unifi
		the same connection)			
» <code>conn_trace_id</code>	String	TraceId to establish connection with client			
» <code>trace_id</code>	String	TraceId for executing order operation			
<code>data</code>	Object	Signature time in seconds			
» <code>result</code>	Object	Response detail to api			
» <code>errs</code>	Object	It is only available when the request fails			
»» <code>label</code>	String	Denotes error type in string format			
»» <code>message</code>	String	Detailed error message			

Order Status

```
futures.order_status
```

You can query an order with this channel and event `api` .

function as `api` below:



Order Status Request

Payload format:

Field	Type	Required	Descr
req_id	string	Yes	Request ID which is sent to the server to help identify the request. The server response will contain this ID from the outside.
req_param	object	Yes	Detail

req_param API order model JSON data:

Field	Type	Required	Descr
order_id	string	Yes	Order ID returned when the order was successfully created by the user. Specific custom field (i.e. the req_id field).

Code example: Login is required before making requests

py

```
import time
import json

# pip install websocket_client
from websocket import create_connection

ws = create_connection("wss://fx-ws.gate.io")

api_status_order = {
    "order_id": "74046543"
}

ws.send(json.dumps({
    "time": int(time.time()),
    "channel": "futures.order_status",
    "event": "api",
    "payload": {
        "req_id": "1ewq-3123w-5",
        "req_param": api_status_order
    }
}))

print(ws.recv())
```

Client request example

json

```
{
  "time": 1681196535,
  "channel": "futures.order_status",
  "event": "api",
  "payload": {
    "req_id": "request-id-2",
    "req_param": {
      "order_id": "74046543"
    }
  }
}
```

Order Status Notification

Result format:

Field	Type	Description
request_id	String	Unique identifier of the message
header	Map	Response meta info
» response_time	String	Response send time in mill
» channel	String	Request channel
» event	String	Request event
» client_id	String	Unique client id
» x_in_time	Integer	time to receive the request (in microseconds)
» x_out_time	Integer	time to return response (in microseconds)
» conn_id	String	Connection ID established with the client (remains consistent for the same connection)

Order status notification example

json

```
{
  "request_id": "request-id-2",
  "header": {
    "response_time": "1681196535985",
    "status": "200",
    "channel": "futures.order_status",
    "event": "api",
    "client_id": "::1-0x14002cfa0c0",
    "x_in_time": 1681985856667508,
    "x_out_time": 1681985856667598,
    "conn_id": "5e74253e9c793974",
    "conn_trace_id": "1bde5aaa0acf2f5",
    "trace_id": "e410abb5f74b4afc519e"
  },
  "data": {
    "result": {
      "id": 74046543,
      "user": 6790020,
      "create_time": 1681196535.01,
      "status": "open",
      "contract": "BTC_USDT",
      "size": "10",
      "price": "31403.2",
      "tif": "gtc",
      "left": "10",
      "fill_price": "0",
      "text": "t-my-custom-id",
      "tkfr": "0.0003",
      "mkfr": "0",
      "stp_id": 2,
      "stp_act": "cn",
      "amend_text": "-"
    }
  }
}
```

Spot &
MarginPerpetual
FuturesDelivery
Futures

Options

Unifi

» <code>conn_trace_id</code>	String	TraceId to establish connection with client
» <code>trace_id</code>	String	TraceId for executing order operation
<code>data</code>	Object	Signature time in seconds
» <code>result</code>	Object	Response detail to api 
» <code>errs</code>	Object	It is only available when the request fails
»» <code>label</code>	String	Denotes error type in string format
»» <code>message</code>	String	Detailed error message

Last Updated: 1/7/2026, 6:30:35 AM